

Beyond Data Scaling: Representation-Centric Pre-training for Vision-Language-Action Models

Abstract

Scaling robot data is crucial for building generalist Vision-Language-Action (VLA) models, yet robot trajectories are fundamentally harder to scale than web-scale image-text data because they require embodied collection and sparsely cover the physical world. This makes representation quality a central bottleneck: under a fixed robot-data budget, VLA pre-training must convert limited trajectories into transferable visual-action knowledge, rather than merely fit actions. We propose **VLAct**, a VLA-oriented VLM backbone trained with a representation-centric pre-training recipe. VLAct preserves the broad VLM prior, avoids over-specializing the backbone to a single action head, and encourages shared action semantics across embodiments through VLM-prior preservation, multi-head continuous action co-supervision, and a partially unified cross-embodiment action layout, while leaving downstream users free to attach task-specific action heads during fine-tuning. Across multi-embodiment simulation benchmarks, real-world robot experiments, and unseen-embodiment transfer, VLAct consistently improves downstream performance under fixed fine-tuning protocols. On LIBERO-Plus and RoboTwin 2.0, VLAct surpasses large-scale industrial VLA systems such as ABot-M0 and LingBot-VLA, achieving 82.6% and 92.5% success, respectively. On the recently published RoboDojo simulation benchmark, VLAct achieves a 10.66 average score and 7.60% success rate, ranking sixth among all policies by success rate. It outperforms all explicitly designated world-action-model (WAM) entries on both metrics, as well as several industry-developed systems. Most notably, on RoboCasa-GR1, a humanoid embodiment never seen during pre-training, **VLAct with only 20% of downstream trajectories already outperforms the full-data GR00T-N1.6 baseline**. These results are obtained using fully open-source data and only a **16-GPU** training setup, showing that representation-centric pre-training can deliver highly competitive performance under a modest compute budget and is an important independent axis of VLA progress beyond data scaling. All models and training pipelines will be open-sourced.

Date: August 2026

Project Page: <https://starvla.github.io/VLAct>

Highlights at a Glance

- H1. Representation-centric pre-training.** VLAct preserves VLM priors, diversifies action-head supervision, and shares action semantics across embodiments. §3
- H2. Strong across benchmarks.** VLAct reaches 82.6% on LIBERO-Plus, 53.4% on VLA-Arena, and 92.5% on RoboTwin 2.0. §4
- H3. Data-efficient embodiment transfer.** With only 20% of RoboCasa-GR1 trajectories, VLAct reaches 49.5%, exceeding full-data GR00T-N1.6 at 47.6%. §4.4
- H4. Open and efficient scaling.** With open data and 16 GPUs, VLAct ranks sixth of 35 policies by RoboDojo success and beats every designated WAM on both metrics. Tab. 3

Contents

1	Introduction	3
2	Pilot Study: How Action Supervision Shapes VLA Backbones	4
3	VLAct: Representation-Centric VLA Pre-training	5
3.1	Overall Pre-training and Fine-tuning Recipe	5
3.2	Preserving the VLM’s Vision-Language Representation	6
3.3	Aligning and Diversifying the Action Representation	6
3.4	Unifying the Action Representation across Embodiments	7
4	Experiments	8
4.1	Effectiveness on Single-arm Franka Manipulation	8
4.2	Effectiveness on Bimanual AgileX Manipulation	8
4.3	Real-world Experiments	9
4.4	Does VLAct Transfer to Unseen Embodiments?	10
5	Related Work	10
6	Conclusion	10
A	Additional Related Work	18
B	Additional Main Results	19
B.1	Additional Results on Franka	19
B.2	Additional Results on Agilex	19
C	Effectiveness of Shallow-Layer Protection During Pre-training	19
C.1	Motivation and Design	19
C.2	Ablation Results	20
D	VLM Co-training for Representation Preservation	20
D.1	Why Mix Auxiliary Data During VLA Pre-training?	20
D.2	Auxiliary Data Mixture	21
D.3	Effect of Different Auxiliary Co-training Data	22
E	Representation Effects of Head-Diverse Pre-training	23
E.1	Head-Diverse Pre-training Prevents Decoder Lock-in	23
E.2	Does Head Diversity Also Improve Same-head Adaptation?	24
F	Unified Joint Space and Wrap-Aware Loss	25
F.1	Unified joint space by wrapping.	25
F.2	Experiments.	26
G	Details of Data Cleaning	26
H	Details of Action Heads	27
H.1	Representative Action-Head Families	27
H.2	Unified Action Representation	29
I	Details of Real-world Experiments	29
I.1	Hardware and observation.	29
I.2	Detailed Real-World Results and Failure Analysis	30
I.3	Detailed Prompts for Each Real-World Robotic Arm Task	31
I.4	Visualization	32
J	Future Work and Broader Impact.	32
	Authors and Advisors	36

1 Introduction

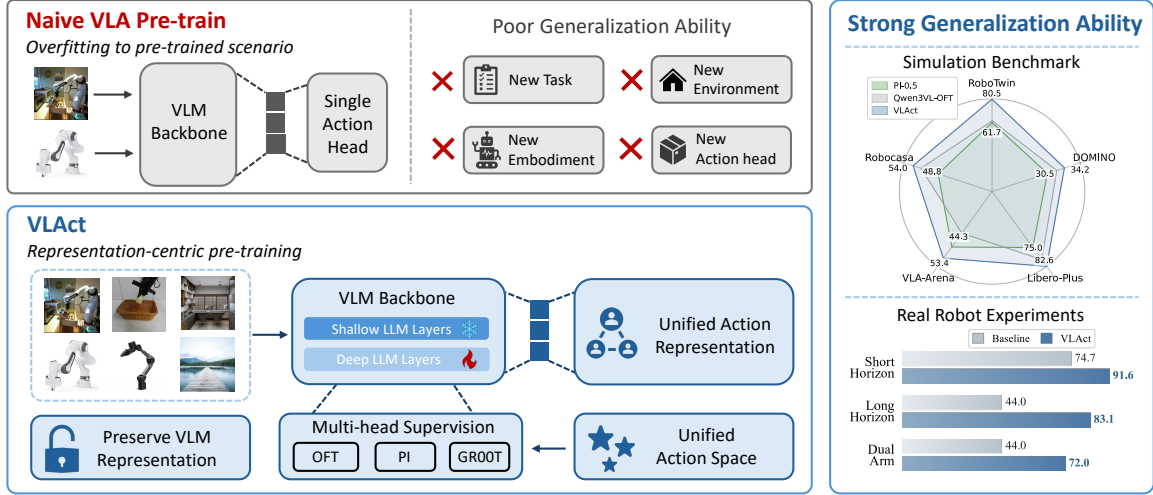


Figure 1: **Overview of VLAct.** VLAct adopts representation-centric pre-training to learn generalizable action-aware representations across tasks, embodiments, environments, and action heads.

Scaling data, model size, and training compute has been a central driver behind modern foundation models Achiam et al. (2023); Team et al. (2023). In language and vision, this scaling paradigm succeeds not simply because web-scale corpora are large Liu et al. (2024); Dai et al. (2023); Bai et al. (2025), but because they provide broad coverage of visual and semantic variation. Vision-Language-Action (VLA) models naturally aspire to the same paradigm: more robot trajectories should, in principle, support more generalist robot policies Black et al. (2024); Zitkovich et al. (2023); Bjorck et al. (2025); Kim et al. (2024); Yang et al. (2026). Yet robot learning faces a qualitatively different scaling bottleneck.

Robot trajectories cannot be scraped from the web. They must be produced through embodied execution in the physical world, often under human teleoperation or carefully designed data-collection protocols O’Neill et al. (2024); Khazatsky et al. (2024); Wu et al. (2025); Lee et al. (2025). More importantly, the space that robot policies must generalize over is combinatorial and continuous, spanning scenes, objects, task goals, embodiments, and contact-rich dynamics. As a result, even very large robot datasets remain sparse samples of the physical interaction space, with partial and uneven coverage across the situations a robot may encounter.

This does not diminish the value of scale, but it changes the role scale should play. If VLA pre-training cannot count on exhaustive coverage O’Neill et al. (2024); Zitkovich et al. (2023), then its success depends not only on how many trajectories are collected, but on how effectively those trajectories induce transferable visual-action representations. We therefore look *beyond data scaling*: not to replace scale, but to ask what must accompany it. ***Given a fixed robot-data budget, how can VLA pre-training learn strong representations that generalize beyond the trajectories on which they were trained?***

This paper takes a representation-centric view of VLA pre-training. Rather than treating robot pre-training merely as large-scale action fitting, we view it as a process of distilling robot trajectories into reusable visual-action knowledge within the backbone. A useful VLA backbone should not only predict the actions observed in the pre-training data, but also internalize transferable priors about objects, affordances, spatial relations, and action-conditioned physical interactions that downstream policies can reuse across new tasks, scenes, and embodiments Huang et al. (2023); Driess et al. (2023). Effective VLA pre-training therefore requires designing how trajectory data shapes the backbone representation, rather than simply optimizing action prediction on the pre-training distribution.

To build a transferable VLA backbone, we first examine how robot pre-training shapes the learned representation, rather than only its action-prediction accuracy. We identify three failure modes in naive VLA pre-training. First, because robot trajectories are much narrower than the web-scale corpora used to train VLMs, full end-to-end updating can erode broadly useful vision-language features. Second, supervision from a single action head can over-specialize the backbone to that head’s decoding geometry, limiting transfer to alternative

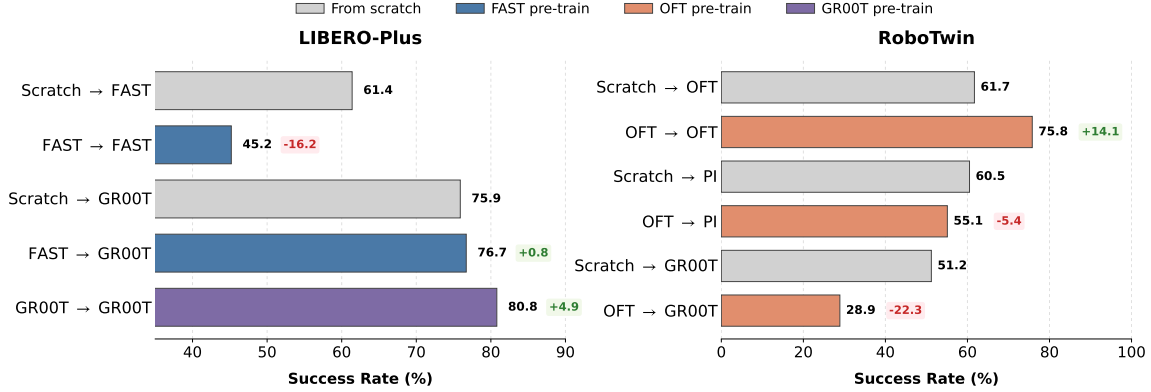


Figure 2: **Action supervision reshapes VLA backbone representations.** Keeping the VLM backbone fixed, we vary the action head used during pre-training and fine-tuning. Discrete FAST supervision can transfer to a continuous GR00T head, but retaining the FAST head suffers from discretization-induced information loss. Continuous OFT supervision improves same-head fine-tuning, but transfers poorly to other continuous heads, suggesting head-specific representation collapse rather than a generally reusable action representation.

heads. Third, embodiment-specific output spaces can weaken cross-embodiment sharing by representing physically comparable actions, such as gripper opening and closing, through isolated robot-specific heads.

Motivated by these observations, we propose **VLAct**, a VLA-oriented VLM backbone trained with a representation-centric pre-training recipe. VLAct preserves the VLM prior through shallow-layer protection and caption-data mixing, prevents single-head over-specialization through multi-head continuous action co-supervision, and encourages shared action semantics through a partially unified cross-embodiment action layout with a wrap-aware loss for periodic joints. These components are used only during pre-training to shape the backbone representation, while downstream users remain free to attach the action head best suited to their task and fine-tune under their own data budget.

We evaluate **VLAct** across multi-embodiment simulation benchmarks, real-world robot experiments, and unseen-embodiment transfer. To isolate the effect of backbone representation quality, we keep the action head, data budget, and fine-tuning recipe fixed, and replace only the pretrained backbone. Under this controlled protocol, VLAct consistently improves over generic VLM backbones and naive VLA-pretrained baselines.

VLAct outperforms strong public and industrial VLA systems across benchmarks: 82.6% on LIBERO-Plus Fei et al. (2025), 53.4% on VLA-Arena Zhang et al. (2025), and 92.5% on RoboTwin 2.0 Chen et al. (2025b). On recent RoboDojo Chen et al. (2026), it achieves a 10.66 score and 7.60% success rate, ranking sixth by success and outperforming all explicitly designated WAM models. With only 20% of RoboCasa-GR1 data, it also surpasses the full-data GR00T-N1.6 baseline Bjorck et al. (2025). All these results are obtained using fully open-source data and only a **16-GPU** training setup, highlight pre-training design as an independent axis of VLA progress.

2 Pilot Study: How Action Supervision Shapes VLA Backbones

A central goal of VLA pre-training is to produce a reusable backbone, rather than a policy initialization tied to one particular action head. In practice, there is no universally optimal action head: discrete token heads (FAST), regression heads (OFT), flow-matching heads (PI), and diffusion-style continuous heads (GR00T) impose different inductive biases and may be preferred under different tasks, embodiments, horizons, or deployment constraints. A useful pretrained backbone should therefore expose action-relevant information in a form that downstream heads can reuse, instead of encoding it in a representation specialized to the head used during pre-training.

As shown in Fig. 2, we isolate this action-head axis through controlled pilot experiments on LIBERO-Plus and the RoboTwin-Clean setting. Fixing the VLM backbone to Qwen3-VL-4B, we vary the action head used during pre-training and fine-tuning, and test how different heads reshape the resulting backbone. We use representative discrete and continuous heads, and the head definitions are shown in Appendix H.1.

Discrete supervision can transfer, but suffers from discretization-induced information loss. The LIBERO-Plus probe asks whether discrete FAST-token pre-training provides sufficient action information for downstream

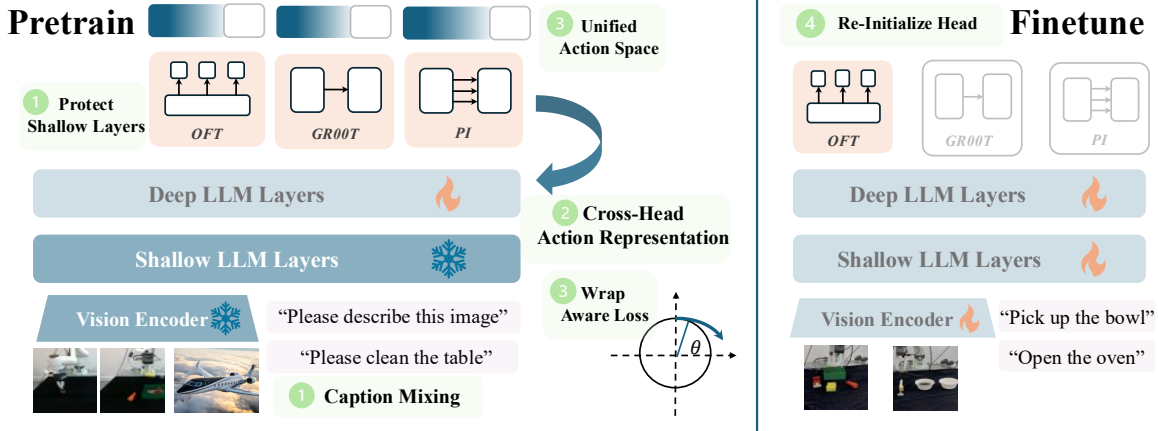


Figure 3: **Overview of VLAct.** During pre-training, ① VLAct preserves the VLM prior through shallow-layer protection and caption mixing (§ 3.2), ② uses multi-head continuous supervision to avoid head-specific representation collapse (§ 3.3), and ③ aligns cross-embodiment action semantics with a unified action layout and wrap-aware loss (§ 3.4). During fine-tuning, ④ we reuse the pretrained backbone and freshly initialize the task-specific action head (§ 3.1).

control. FAST supervision can inject transferable structure into the backbone: when the FAST-pretrained backbone is paired with a continuous GR00T head, performance slightly improves over from-scratch GR00T fine-tuning. However, retaining the FAST Action Head performs much worse than continuous-head fine-tuning, and FAST pre-training does not close this gap. This suggests that discrete action tokens teach coarse action structure, but lose fine-grained temporal and amplitude information that matters for manipulation.

Single-head continuous supervision can induce head-specific representation collapse. The RoboTwin-Clean probe shows a different failure mode. OFT pre-training substantially improves downstream performance when the fine-tuning head is also OFT, confirming that continuous action supervision can inject fine-grained action information into the backbone. However, this gain does not transfer to other continuous heads: the same OFT-pretrained backbone performs poorly when paired with PI or GR00T. This suggests that the learned representation is not simply more action-aware in a head-agnostic sense. Instead, single-head pre-training can collapse the backbone feature geometry toward the directions that are most useful for the pre-training head. Action-relevant information may remain present, but it is organized in a head-specific form that other downstream heads cannot easily decode. Strong same-head performance therefore overstates backbone reusability.

Analysis. Action supervision is not neutral: the pre-training head directly shapes the backbone representation. Discrete supervision can transfer across heads, but loses fine-grained action information through discretization. Continuous supervision avoids this bottleneck, but single-head pre-training can cause head-specific representation collapse, organizing action features around one head geometry rather than keeping them broadly accessible. Thus, a transferable VLA backbone should preserve fine-grained action information while supporting multiple downstream heads. This motivates VLAct’s multi-head continuous co-supervision in Sec. 3.3.

3 VLAct: Representation-Centric VLA Pre-training

3.1 Overall Pre-training and Fine-tuning Recipe

As shown in Fig. 3, VLAct pre-trains a reusable VLA backbone from a VLM initialization. During pre-training, it preserves the VLM prior by freezing the vision encoder and shallow LLM layers, while mixing robot trajectories with caption data. Robot samples supervise the shared backbone through multiple continuous action heads, including OFT, PI, and GR00T, under a partially unified action layout with masked inactive dimensions and wrap-aware losses.

These pre-training components are used only to shape the backbone. During fine-tuning, we discard the pre-training heads and caption stream, attach a freshly initialized task-specific action head, and train under the same downstream setting as each baseline. Thus, improvements come from the learned backbone representation rather than from reusing a pre-adapted action head.

3.2 Preserving the VLM’s Vision-Language Representation

Motivation. A VLA backbone starts from a strong vision-language model, such as Qwen3-VL, whose visual and language representations are learned from broad web-scale data. Naive action pre-training can overwrite this prior: robotics trajectories are expensive, narrow in visual diversity, and concentrated on a limited set of manipulation scenes. As a result, end-to-end VLA pre-training may improve action prediction on the pre-training distribution while degrading the general vision-language features that make the backbone transferable. Our first design goal is therefore to inject action knowledge without unnecessarily drifting away from the original VLM representation. We use two simple mechanisms: shallow-layer protection and caption-mixed pre-training.

Shallow-layer protection. We freeze the entire vision encoder and the lower half of the LLM layers during VLA pre-training, while updating the upper LLM layers and the action heads. The motivation is to protect the parts of the model most responsible for low-level visual processing and early vision-language alignment, while still allowing the upper layers to adapt to action-conditioned reasoning. During downstream fine-tuning, we unfreeze the full model so that all parameters can adapt to the target task. This simple freezing strategy improves downstream performance by 3.7% on LIBERO-Plus and 3.4% on Agilex. Detailed ablations are provided in Appendix C.

Caption-mixed pre-training. Mixing VLM data during VLA pre-training is a common strategy for mitigating representation drift, but not all VLM supervision is equally useful. We ablate several VLM data mixtures and find that captioning data provides the strongest anchor for downstream performance. The reason is intuitive: captions offer dense supervision over objects, attributes, spatial relations, and scene context, helping preserve the backbone’s original vision-language representation while it learns from robot trajectories. We therefore use caption-mixed pre-training in VLAct. Detailed ablations are provided in Appendix D.

Takeaway 1. *VLAct preserves the VLM prior while injecting action knowledge: shallow-layer protection limits representation drift, and caption mixing anchors the trainable layers with dense vision-language supervision.*

3.3 Aligning and Diversifying the Action Representation

Design principle. The pilot study in Sec. 2 suggests that an effective VLA backbone should satisfy two requirements. First, it should support *head transfer*: downstream users should be able to attach action heads suited to their target embodiments and deployment scenarios, without inheriting a representation specialized to one particular head geometry. Second, it should learn high-quality action features, rather than merely fitting the pre-training head. Naive single-head pre-training can violate these requirements. Since the backbone and the attached head are optimized jointly, the backbone may encode action information in a form tailored to that head. This head-specific feature specialization reduces transferability across heads, even if the resulting representation performs well when the same head is reused.

Co-supervised multi-head pre-training. We address this by pre-training the same backbone with multiple action heads in parallel. Concretely, we attach three representative continuous heads, *OFT*, *PI*, and *GR00T*, to the shared VLA backbone. Given the same vision-language input, the backbone produces a shared latent representation z . Each head receives the same z and predicts the same ground-truth action chunk a . The training objective is

$$\mathcal{L}_{\text{action}} = \mathcal{L}_{\text{OFT}} + \mathcal{L}_{\text{PI}} + \mathcal{L}_{\text{GR00T}}.$$

This objective prevents the representation from becoming tied to a single decoder: the backbone must expose action information in a form that can be used by several action parameterizations. Since all heads share the same backbone forward pass, multi-head supervision adds only lightweight head-specific computation rather than repeated backbone computation.

This design is deliberately simple: we do not introduce a new action head or a specialized alignment module. Instead, we use head diversity itself as supervision. Because the three heads impose different objectives and decoder biases on the same action-prediction problem, the backbone cannot rely on features that are useful only to one head. To reduce all losses simultaneously, it must encode action information in a form that remains accessible across multiple downstream parameterizations.

Co-supervision therefore plays two roles. It makes the backbone more head-agnostic, improving transfer to alternative downstream heads; and it regularizes the representation, yielding stronger action features even when the downstream head matches one of the pre-training heads. As shown in Appendix E, this second effect

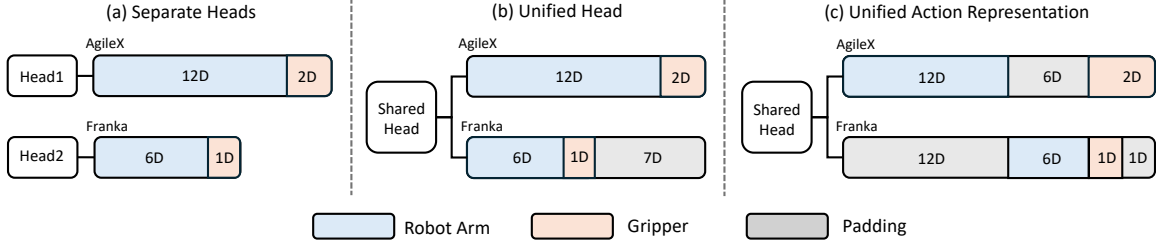


Figure 4: **Unified action space design.** We compare separate embodiment-specific heads, a naively unified action space, and our partially unified action space. VLAct shares physically aligned dimensions, such as gripper open/close, while masking unavailable or embodiment-specific dimensions.

is empirically important: multi-head pre-training not only improves head transfer, but can also outperform single-head pre-training under the same downstream head.

Takeaway 2. *Multi-head co-supervision turns head diversity into a representation-learning signal. By training OFT, PI, and GROOT on the same backbone latent and action targets, VLAct forces action information to remain accessible across different head geometries rather than collapsing toward one of them. This produces a backbone that transfers better across downstream heads and learns stronger action features than single-head pre-training.*

3.4 Unifying the Action Representation across Embodiments

Motivation. The previous Sec. 3.3 addresses variation across action heads. We now consider variation across robot embodiments. Different embodiments have different action spaces, and a common solution is to attach a separate action head or output projector to each robot type. This is flexible, but it also hides any shared action structure inside embodiment-specific heads. Our motivation is simple: when two embodiments share a physically meaningful action dimension, the supervision for that dimension should also be shared; when they do not, we should not force an artificial alignment.

Partially unified cross-embodiment action space. To share action supervision across embodiments without forcing incompatible degrees of freedom into the same coordinates, VLAct uses a partially unified cross-embodiment action space. Fig. 4 compares three natural designs. The first uses embodiment-specific heads, which isolate supervision across robots. The second uses a fully unified action space, where lower-dimensional robots are padded to a common action dimension; this can naively align coordinates with different physical meanings. VLAct adopts the third design: one shared action head with an action space that is unified only along physically comparable dimensions.

Concretely, gripper dimensions are shared across embodiments because open/close commands have comparable semantics, while arm dimensions remain embodiment-specific when the robots have different kinematics or degrees of freedom. During training, each sample contributes loss only on the active dimensions for its embodiment, and inactive dimensions are masked out. This keeps the architecture minimal: no embodiment adapter, routing module, or embodiment-conditioned decoder is introduced. Instead, cross-embodiment sharing is induced directly through shared coordinates in the action space. As shown in Appendix. H.2, this partial unification improves downstream performance by sharing supervision where action semantics are physically aligned while avoiding naive full sharing across incompatible joints.

Wrap-aware loss for periodic joint angles. We also handle a small but important parameterization issue for absolute joint angles. Standard regression treats 179° and -179° as 358° apart, although they are physically only 2° apart. We therefore apply a wrap-aware loss on periodic joint-angle dimensions, measuring angular residuals modulo 360° . This correction is only used for absolute joint dimensions; the full definition and dimension scope are provided in Appendix H.2.

Takeaway 3. *Cross-embodiment pre-training benefits from sharing supervision where semantics are physically aligned. VLAct uses one shared head with a partially unified action space: gripper dimensions are shared across embodiments, while incompatible arm dimensions remain embodiment-specific and masked when inactive. This avoids isolated embodiment-specific heads and naive full sharing, yielding stronger cross-embodiment representations and better downstream performance.*

Table 1: **Results on LIBERO-Plus.** Per-dimension success rate (%) on the LIBERO-Plus robustness benchmark. All models are trained on standard LIBERO and evaluated on the perturbed test set. Dark blue highlights VLAct and per-column best results; light blue highlights the strongest prior method by total score and per-column second-best results.

Method	Camera	Robot	Lang.	Light	Bg.	Noise	Layout	Total
OpenVLA (Kim et al., 2024)	0.8	3.5	23.0	8.1	34.8	15.2	28.5	15.6
OpenVLA-OFT (Kim et al., 2025)	56.4	31.9	79.5	88.7	93.3	75.8	74.2	69.6
NORA (Hung et al., 2025)	2.2	37.0	65.1	45.7	58.6	12.8	62.1	39.0
WorldVLA (Cen et al., 2025)	0.1	27.9	41.6	43.7	17.1	10.9	38.0	25.0
UniVLA (Wang et al., 2025)	1.8	46.2	69.6	69.0	81.0	21.2	31.9	42.9
π_0 (Black et al., 2024)	13.8	6.0	58.8	85.0	81.4	79.0	68.8	53.6
π_0 -FAST (Pertsch et al., 2025)	65.1	21.6	61.0	73.2	73.2	74.4	68.8	61.6
RIPT-VLA (Tan et al., 2025)	55.2	31.2	77.5	88.3	91.6	73.5	74.2	68.4
Abot-M0 (Yang et al., 2026)	60.4	67.9	86.4	96.2	91.6	86.4	82.6	80.5
Qwen3VL-OFT	47.0	60.1	87.0	96.3	95.3	73.1	79.2	75.0
VLAct	73.9	68.4	81.5	96.7	96.7	86.0	83.3	82.6

4 Experiments

VLAct is pre-trained on fully open-source robot datasets, including DROID (Khazatsky et al., 2024), InternA1 (Tian et al., 2025), RoboCoin (Wu et al., 2025), and MolmoAct (Lee et al., 2025), together with captioning data for VLM representation preservation. Data processing and cleaning details are provided in Appendix G. We build on the StarVLA training codebase (Community, 2026) and use Qwen3-VL-4B as the base vision-language backbone. All experiments are conducted on 16 GPUs. We will release the training scripts and model checkpoints.

4.1 Effectiveness on Single-arm Franka Manipulation

Evaluation benchmarks. We evaluate single-arm Franka manipulation mainly on *LIBERO-Plus* (Fei et al., 2025), a recent benchmark designed to probe robustness beyond the saturated standard LIBERO setting. LIBERO-Plus introduces systematic test-time perturbations, including changes in camera views, robot states, visual noise, object layouts, and task instructions, making it suitable for evaluating transferable VLA pre-training. We compare VLAct with representative VLA systems, including π_0 (Black et al., 2024), $\pi_{0.5}$ (Intelligence et al., 2025), and Abot-M0 (Yang et al., 2026). Baseline numbers are taken from LIBERO-Plus (Fei et al., 2025). Additional Franka results, VLA-Arena, are reported in Appendix B.1.

LIBERO-Plus results. Table 1 shows that our VLAct achieves the best overall score, reaching **82.6%**, and improves over our strongest in-house baseline, Qwen3VL-OFT, by **7.6%** (82.6% vs. 75.0%). This comparison isolates the effect of the pre-training recipe, since both models use the same Qwen3-VL backbone family and an OFT downstream head. VLAct also surpasses Abot-M0, a large-scale VLA system from Alibaba, by 2.1%, using fully open-source data and substantially fewer training resources. The largest gains over Qwen3VL-OFT appear on *Camera*, *Robot*, *Noise*, and *Layout*, suggesting that representation-centric pre-training yields more robust visual-spatial representations than standard Qwen3-VL fine-tuning on LIBERO.

4.2 Effectiveness on Bimanual AgileX Manipulation

Evaluation benchmarks. To evaluate bimanual AgileX manipulation, we use two recent benchmarks with complementary generalization axes: *RoboTwin 2.0* (Chen et al., 2025b) and *DOMINO* (Fang et al., 2026). RoboTwin 2.0 evaluates robust dual-arm manipulation under clean scenes and randomized visual conditions. We report two settings: the *Base* setting uses only 50 clean trajectories per task, while the *Data Scaling* setting further adds 500 random trajectories per task. This allows us to evaluate both performance under limited clean trajectories and whether the pre-trained model can continuously benefit from more downstream robot trajectories. Moreover, DOMINO evaluates dynamic manipulation with moving objects and changing environments, targeting the spatiotemporal reasoning abilities often missed by static manipulation benchmarks. Additional results are shown in Sec. B.2.

RoboTwin 2.0 results. Table 2 shows that VLAct achieves state-of-the-art performance in both the *Base* and *Data Scaling* settings in *Clean* and *Random* set evaluation. In the *Base* setting, VLAct is fine-tuned

Table 2: **RoboTwin 2.0 results.** Task success rate (%) under the Clean and Random regimes, where Random introduces variations in object identity, pose, lighting, and texture. We report results under two downstream training settings: Base, which uses 50 clean trajectories per task, and Data Scaling, which adds 500 randomized trajectories per task. Each regime contains 50 tasks with 100 episodes per task. Dark blue highlights the strongest VLAct configuration; light blue highlights alternative VLAct action heads. Best results are bold and second-best results are underlined.

Method	Action Head	Base Setting		Data Scaling Setting	
		Clean	Random	Clean	Random
<i>Continuous heads</i>					
π_0 (Black et al., 2024)	PI	46.4	16.4	65.9	58.4
$\pi_{0.5}$ (Intelligence et al., 2025)	PI	60.2	-	82.7	76.8
X-VLA (Zheng et al., 2025)	PI	-	-	72.9	72.8
Lingbot-VLA (Wu et al., 2026)	PI	-	-	88.6	86.7
Abot-M0 (Yang et al., 2026)	PI	-	-	86.1	85.1
Qwen3VL-OFT	OFT	61.7	10.5	88.2	88.3
<i>VLAct (same backbone, varying continuous head)</i>					
VLAct	OFT	80.5	41.5	92.5	90.8
VLAct	GR00T	74.2	22.9	<u>89.6</u>	87.4
VLAct	PI	<u>78.4</u>	<u>24.1</u>	89.4	<u>88.8</u>

only with clean trajectories, yet still performs strongly on the *Random* evaluation set. This indicates that the pretrained backbone improves not only sample-efficient adaptation, but also clean-to-random generalization under distribution shifts in visual appearance and scene configuration. In the *Data Scaling* setting, VLAct remains strong as more downstream data is added, showing that the learned representation is not merely tuned to a low-data regime. Across downstream head choices, the performance remains stable, supporting the head-transfer claim in Sec. 3.3: the transferred object is the backbone representation, not a particular action head.

4.3 Real-world Experiments

In what follows, we perform the real-world robot experiments using Franka Research 3 7-DoF arms to validate the effectiveness of our VLAct. We evaluate the proposed VLAct and a competitive baseline (Qwen3VL-4B-OFT without pre-training) across four regimes: **(1) In-Domain (ID) single-arm short-horizon tasks**, such as ‘Carrot-from-pot’, ‘Button pressing’, ‘Cube stacking’, ‘Pen in cup’; **(2) ID single-arm long-horizon tasks**, e.g., ‘Table cleaning’ & ‘Scoop beans’; **(3) ID dual-arm coordination**, with tasks ‘Unplugging’, ‘Breakfast preparation’, ‘Banana handover-place’, ‘Fold pants’, ‘Fold towel’; and **(4) Out-of-Domain (OOD) generalization**, including novel objects from pot, novel objects in cup, and table cleaning with extended or fully substituted object types.

We train two separate models for their respective evaluations: one for all single-arm tasks while another for all dual-arm tasks. Each model is trained with 50k training steps on 8 H800 GPUs, and evaluated over 10 trials to calculate the success rate. Detailed hardware settings, prompts, visualizations of each task, and success rate calculation criterion are given in Appendix I.

Results. As shown in Fig. 5, VLAct consistently improves real-world performance over the non-pre-trained Qwen3VL-4B-OFT baseline. In single-arm short-horizon ID tasks (Fig. 5 (a)), VLAct achieves 92.5% average success, higher than the baseline’s 77.5%, with clear gains on spatially sensitive tasks such as cube stacking and pen-in-cup. VLAct also generalizes better to novel objects: on the short-horizon OOD tasks, it obtains 90.0% success on both ‘novel object from pot’ and ‘novel object in cup’, compared with 73.3% and 65.0% for the baseline. In single-arm long-horizon tasks (Fig. 5 (b)), VLAct achieves weighted scores of 86.6% on table cleaning and 80.0% on scooping beans, outperforming the baseline scores of 73.3% and 33.3%. Under harder OOD long-horizon settings, VLAct keeps high scores of 82.5% for extended-sequence table cleaning and 83.3% for full object substitution, while the baseline drops to 47.5% and 46.6%. Finally, despite being pre-trained only on single-arm data, VLAct transfers effectively to dual-arm coordination, reaching 72.0% average success across dual-arm tasks versus 44.0% for the baseline, with the largest gains on deformable-object and synchronization-heavy tasks such as pants folding and towel folding (Fig. 5 (c)). These results suggest that

VLAct improves not only in-domain manipulation accuracy, but also novel-object generalization, long-horizon execution, and dual-arm coordination.

4.4 Does VLAct Transfer to Unseen Embodiments?

Setup. We test whether VLAct learns action representations that transfer to embodiments unseen during pre-training. VLAct is pre-trained only on Franka single-arm and AgileX bimanual data; both the GR-1 humanoid and the ARX X5 bimanual platform are held out during pre-training. We fine-tune VLAct-OFT on RoboCasa-GR1 under a fixed downstream protocol and additionally evaluate VLAct through the official RoboDojo simulation benchmark on ARX X5. These settings directly probe cross-embodiment transfer because the target robots differ from the pre-training embodiments in morphology, action space, and manipulation dynamics.

RoboCasa-GR1 results. Fig. 5(d) reports RoboCasa-GR1 performance under different fractions of the downstream training data. With full fine-tuning data, VLAct reaches **54.0%**, outperforming the strongest reported full-data baselines, including Qwen3VL-OFT at 48.8%, GR00T-N1.6 at 47.6%, and $\pi_{0.5}$ at 37.0%. Since GR-1 trajectories are never used in VLAct pre-training, this result shows that the backbone learned from Franka and AgileX can provide a stronger initialization for a new humanoid embodiment.

The data-efficiency curve further supports this conclusion. With only 20% of the RoboCasa-GR1 trajectories, VLAct already reaches 49.5%, matching or exceeding the full-data Qwen3VL-OFT baseline. With 50% of the data, performance rises to 51.0%, and full-data fine-tuning further improves it to 54.0%. Thus, VLAct is not merely benefiting from downstream data scale; the pretrained backbone transfers useful action structure to the unseen embodiment before full task-specific adaptation. These results support the central claim that VLAct learns reusable action representations rather than overfitting to the robot embodiments observed during pre-training.

RoboDojo results. RoboDojo (Chen et al., 2026) evaluates policies on 42 ARX X5 simulation tasks spanning Generalization, Precision, Long-Horizon, Memory, and Open capabilities. Each task is evaluated over 50 episodes, and the benchmark reports both a partial-progress score and binary task success. Table 3 reports the official leaderboard snapshot from August 24, 2026. Among 35 policies, VLAct ranks **eighth by average score** and **sixth by average success rate**, placing it in the top quartile under both metrics. Among the four explicitly designated world-action-model (WAM) entries, X-WAM is the strongest, with a 7.69 score and 3.83% success rate. VLAct exceeds X-WAM by **2.97 score points** and **3.77 percentage points** in success, thereby outperforming every designated WAM entry on both aggregate metrics. VLAct also outperforms several industry-developed systems, including Xiaomi-Robotics-0, GalaxeaVLA (G0), LingBot-VLA (Wu et al., 2026), and ABot-M0 (Yang et al., 2026). Moreover, six of the seven higher-scoring entries are submitted by industry teams or robotics companies: Dexmal, Galaxea AI, Xiaomi Robotics, Tencent Robotics X, OpenHelix Robotics, and Physical Intelligence. Although the leaderboard does not normalize for training compute, this result is notable given that VLAct uses fully open-source data and a 16-GPU pre-training setup.

Compared with the Qwen3-VL-based StarVLA- α (Ye et al., 2026) entry, VLAct improves the average score by 4.26 points and success by 4.36 percentage points. The largest gains occur on Precision and Long-Horizon tasks, while Memory remains a clear limitation. Since ARX X5 is also absent from VLAct pre-training, these results provide an additional cross-embodiment test under a broad, externally defined evaluation suite.

5 Related Work

Generalist Robot Policies. Recent research has shifted toward generalist policies via Vision-Language-Action models Brohan et al. (2022); Kim et al. (2024); Zitkovich et al. (2023). By fine-tuning web-scale vision-language models Beyer et al. (2024) on diverse robotic datasets O’Neill et al. (2024); Khazatsky et al. (2024), VLAs exhibit remarkable generalization across novel instructions and environments. However, frontier models (e.g., the π series Black et al. (2024)) often rely on closed-source data, hindering reproducibility. Our work provide a systematic recipe study using only public data, focusing on the representation learning nuances that determine VLA performance. The additional related work is shown in Appendix A.

6 Conclusion

We presented **VLAct**, a representation-centric VLA pre-training recipe for building more transferable VLM backbones from limited robot data. Instead of treating robot pre-training as large-scale action fitting, VLAct is

Table 3: **RoboDojo simulation results.** Top 20 policies from the official August 24, 2026 leaderboard, ordered by average score. Each cell reports partial-progress score / success rate (%). Gen.-Std. and Gen.-Rand. denote the standard and randomized Generalization settings. The full leaderboard contains 35 policies; VLAct ranks eighth by score and sixth by success rate. The **dark-blue row** highlights VLAct; **light-blue rows** and [†] identify explicitly designated WAM entries.

Method	Gen.-Std.	Gen.-Rand.	Precision	Long-Horizon	Memory	Open	Average
DM0.5	23.49 / 18.00	8.06 / 4.00	24.82 / 16.75	33.70 / 19.50	47.74 / 47.44	2.43 / 2.08	24.90 / 19.34
GalaxeaVLA (G0.5)	26.74 / 20.00	11.16 / 6.00	28.25 / 20.42	44.12 / 32.25	8.61 / 7.33	1.73 / 1.58	20.23 / 14.88
Xiaomi-Robotics-1	35.65 / 28.00	11.44 / 6.00	26.69 / 18.83	38.39 / 23.67	7.81 / 6.56	3.94 / 3.58	20.07 / 13.93
Hy-Embodied-0.5-VLA	21.98 / 17.00	1.57 / 0.00	13.81 / 8.00	25.74 / 14.92	13.37 / 12.11	0.65 / 0.58	13.07 / 8.80
Spatial Forcing	21.25 / 15.00	6.98 / 4.00	17.33 / 10.58	23.26 / 14.58	5.43 / 4.11	1.78 / 1.58	12.38 / 8.04
$\pi_{0.5}$	20.93 / 15.00	5.82 / 1.00	12.40 / 5.50	23.54 / 14.67	5.78 / 4.56	1.98 / 1.67	11.41 / 6.91
InternVLA-A1.5	16.81 / 12.00	3.90 / 2.00	15.23 / 10.17	23.80 / 13.75	4.93 / 3.56	1.43 / 1.42	11.15 / 7.14
VLAct	16.33 / 12.00	2.74 / 1.00	20.62 / 15.25	20.12 / 13.67	0.66 / 0.56	2.37 / 2.25	10.66 / 7.60
X-VLA	17.90 / 12.00	3.04 / 1.00	18.32 / 12.00	16.53 / 9.75	4.76 / 3.56	0.55 / 0.50	10.13 / 6.52
X-WAM [†]	11.24 / 5.00	3.54 / 1.00	6.72 / 1.83	17.47 / 9.08	6.32 / 4.67	0.57 / 0.25	7.69 / 3.83
Xiaomi-Robotics-0	13.81 / 11.00	1.05 / 0.00	8.42 / 4.58	13.51 / 6.92	5.07 / 3.67	0.22 / 0.17	6.93 / 4.18
StarVLA- α (Ye et al., 2026)	7.54 / 5.00	0.33 / 0.00	9.90 / 4.33	14.15 / 6.50	3.34 / 2.44	0.68 / 0.58	6.40 / 3.24
GigaWorld-Policy-0 [†]	10.28 / 6.00	0.41 / 0.00	6.15 / 1.83	15.51 / 8.92	3.46 / 2.22	0.54 / 0.50	6.20 / 3.27
GalaxeaVLA (G0)	8.71 / 6.00	0.36 / 0.00	8.10 / 3.83	12.60 / 5.58	3.17 / 1.89	0.70 / 0.67	5.82 / 2.96
LingBot-VLA	10.88 / 8.00	2.55 / 1.00	5.33 / 1.83	10.89 / 5.25	3.82 / 2.78	0.72 / 0.67	5.50 / 2.96
EventVLA	6.68 / 3.00	1.22 / 0.00	10.13 / 5.75	5.05 / 0.83	4.92 / 4.78	0.80 / 0.75	4.97 / 2.81
AHA-WAM [†]	10.32 / 6.00	1.26 / 0.00	5.86 / 2.42	8.61 / 2.67	2.97 / 2.78	0.88 / 0.83	4.82 / 2.39
ABot-M0	9.20 / 5.00	2.26 / 2.00	5.50 / 1.75	3.96 / 0.50	2.44 / 2.22	0.72 / 0.67	3.67 / 1.73
Fast-WAM [†]	4.33 / 2.00	0.34 / 0.00	1.96 / 0.00	9.14 / 5.17	3.55 / 3.44	0.42 / 0.42	3.48 / 2.03
π_0	7.18 / 5.00	0.71 / 0.00	3.56 / 0.75	6.19 / 2.00	3.47 / 2.11	0.25 / 0.25	3.48 / 1.53

designed to preserve the broad VLM prior, avoid over-specialization to a single action head, and encourage shared action semantics across embodiments. Across multi-embodiment simulation benchmarks, real-world robot experiments, and unseen-embodiment transfer, VLAct consistently improves downstream performance under fixed fine-tuning protocols. It surpasses strong VLA baselines on LIBERO-Plus, achieves state-of-the-art performance on RoboTwin 2.0, ranks in the top quartile of the all policy RoboDojo leaderboard while outperforming several industry-developed systems, and rapidly adapts to a humanoid embodiment unseen during pre-training on RoboCasa-GR1. These results are obtained using fully open-source data and only a 16-GPU training setup. Their competitiveness against predominantly industry-backed systems shows that VLA progress is not only a matter of scaling robot trajectories or compute: better pre-training design can substantially improve what a fixed resource budget delivers.

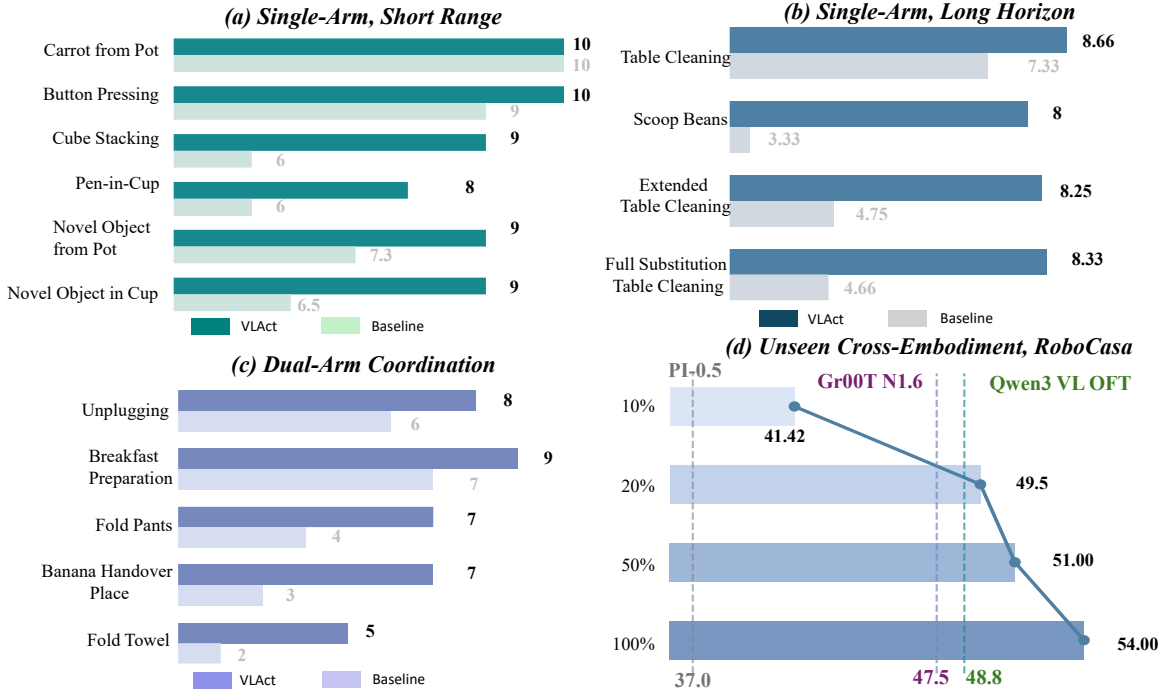


Figure 5: **Real-machine evaluation and cross-embodiment transfer.** (a)–(c) Real-machine evaluation results on single-arm short-range, single-arm long-horizon, and dual-arm coordination tasks, where VLAct consistently improves over the baseline across diverse manipulation settings. (d) Cross-embodiment transfer on RoboCasa-GR1, where VLAct transfers to the held-out GR-1 embodiment and surpasses reported full-data baselines with only 20% downstream data.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. (2023). Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Aldaco, J., Armstrong, T., Baruch, R., Bingham, J., Chan, S., Draper, K., Dwibedi, D., Finn, C., Florence, P., Goodrich, S., et al. (2024). Aloha 2: An enhanced low-cost hardware for bimanual teleoperation. *arXiv preprint arXiv:2405.02292*.
- Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al. (2025). Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*.
- Beyer, L., Steiner, A., Pinto, A. S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschannen, M., Bugliarello, E., et al. (2024). Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*.
- Bharadhwaj, H., Gupta, A., Tulsiani, S., and Kumar, V. (2023). Zero-shot robot manipulation from passive human videos. *arXiv preprint arXiv:2302.02011*.
- Bharadhwaj, H., Mottaghi, R., Gupta, A., and Tulsiani, S. (2024a). Track2act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, pages 306–324. Springer.
- Bharadhwaj, H., Vakil, J., Sharma, M., Gupta, A., Tulsiani, S., and Kumar, V. (2024b). Roboagent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4788–4795. IEEE.
- Bhateja, C., Guo, D., Ghosh, D., Singh, A., Tomar, M., Vuong, Q., Chebotar, Y., Levine, S., and Kumar, A. (2023). Robotic offline rl from internet videos via value-function pre-training. *arXiv preprint arXiv:2309.13041*.
- Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., et al. (2025). Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. (2024). π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*.
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., et al. (2021). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al. (2022). Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*.
- Bu, Q., Cai, J., Chen, L., Cui, X., Ding, Y., Feng, S., Gao, S., He, X., Hu, X., Huang, X., et al. (2025). Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*.
- Cai, Z., Wang, R., Gu, C., Pu, F., Xu, J., Wang, Y., Yin, W., Yang, Z., Wei, C., Sun, Q., et al. (2025). Scaling spatial intelligence with multimodal foundation models. *arXiv preprint arXiv:2511.13719*.
- Cen, J., Yu, C., Yuan, H., Jiang, Y., Huang, S., Guo, J., Li, X., Song, Y., Luo, H., Wang, F., et al. (2025). Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*.
- Chen, G., Shen, L., Shao, R., Deng, X., and Nie, L. (2024a). Lion: Empowering multimodal large language model with dual-level visual knowledge. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26540–26550.
- Chen, J., Wei, F., Zhao, J., Song, S., Wu, B., Peng, Z., Chan, S.-H. G., and Zhang, H. (2025a). Revisiting referring expression comprehension evaluation in the era of large multimodal models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 513–524.
- Chen, L., Li, J., Dong, X., Zhang, P., He, C., Wang, J., Zhao, F., and Lin, D. (2024b). Sharegpt4v: Improving large multi-modal models with better captions. In *European Conference on Computer Vision*, pages 370–387. Springer.
- Chen, T., Chen, Y., Li, Z., Tang, J., Su, K., Lu, H., Wan, W., Chen, B., Liu, S., Yan, H., Su, H., Dou, Z., Wang, K., Zhang, D., Liu, Y., Qin, Y., Liang, Q., Wu, Q., Lin, Z., Lin, W., Wang, Y., He, M., Wu, T., Wu, R., Zhou, J., Lei, K.-C., Yu, H., Ji, Y., Jin, W., Lin, G., Li, X., Xiong, Q., Xu, R., Li, Z., Chai, W., Xie, E., Wang, Z., Mu, Y., Dong, H., Matusik, W., Ding, M., Ding, W., Luo, P., and Tomizuka, M. (2026). RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*.
- Chen, T., Chen, Z., Chen, B., Cai, Z., Liu, Y., Li, Z., Liang, Q., Lin, X., Ge, Y., Gu, Z., et al. (2025b). Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*.

- Chen, Z., Wang, W., Cao, Y., Liu, Y., Gao, Z., Cui, E., Zhu, J., Ye, S., Tian, H., Liu, Z., et al. (2024c). Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*.
- Chi, C., Xu, Z., Pan, C., Cousineau, E., Burchfiel, B., Feng, S., Tedrake, R., and Song, S. (2024). Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. *arXiv preprint arXiv:2402.10329*.
- Community, S. (2026). Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*.
- Dai, W., Li, J., Li, D., Tiong, A., Zhao, J., Wang, W., Li, B., Fung, P. N., and Hoi, S. (2023). Instructblip: Towards general-purpose vision-language models with instruction tuning. *Advances in neural information processing systems*, 36:49250–49267.
- Damen, D., Doughty, H., Farinella, G. M., Fidler, S., Furnari, A., Kazakos, E., Moltisanti, D., Munro, J., Perrett, T., Price, W., et al. (2020). The epic-kitchens dataset: Collection, challenges and baselines. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):4125–4141.
- Deitke, M., Clark, C., Lee, S., Tripathi, R., Yang, Y., Park, J. S., Salehi, M., Muennighoff, N., Lo, K., Soldaini, L., et al. (2025). Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 91–104.
- Doshi, R., Walke, H., Mees, O., Dasari, S., and Levine, S. (2024). Scaling cross-embodied learning: One policy for manipulation, navigation, locomotion and aviation. *arXiv preprint arXiv:2408.11812*.
- Driess, D., Xia, F., Sajjadi, M. S., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., et al. (2023). Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*.
- Fang, H., Fang, H.-S., Wang, Y., Ren, J., Chen, J., Zhang, R., Wang, W., and Lu, C. (2024). Airexo: Low-cost exoskeletons for learning whole-arm manipulation in the wild. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15031–15038. IEEE.
- Fang, H., Li, S., Wang, S., Xi, X., Liang, D., and Bai, X. (2026). Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*.
- Fei, S., Wang, S., Shi, J., Dai, Z., Cai, J., Qian, P., Ji, L., He, X., Zhang, S., Fei, Z., et al. (2025). Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*.
- Grauman, K., Westbury, A., Byrne, E., Chavis, Z., Furnari, A., Girdhar, R., Hamburger, J., Jiang, H., Liu, M., Liu, X., et al. (2022). Ego4d: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18995–19012.
- Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Dong, Y., Ding, M., et al. (2024). Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14281–14290.
- Huang, W., Jia, B., Zhai, Z., Cao, S., Ye, Z., Zhao, F., Xu, Z., Tang, X., Hu, Y., and Lin, S. (2025). Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*.
- Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., and Fei-Fei, L. (2023). Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*.
- Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., et al. (2022). Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*.
- Hung, C.-Y., Sun, Q., Hong, P., Zadeh, A., Li, C., Tan, U., Majumder, N., Poria, S., et al. (2025). Nora: A small open-sourced generalist vision language action model for embodied tasks. *arXiv preprint arXiv:2504.19854*.
- Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., et al. (2025). $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*.
- Jiang, Y., Gupta, A., Zhang, Z., Wang, G., Dou, Y., Chen, Y., Fei-Fei, L., Anandkumar, A., Zhu, Y., and Fan, L. (2023). Vima: Robot manipulation with multimodal prompts.
- Kapelyukh, I., Vosylius, V., and Johns, E. (2023). Dall-e-bot: Introducing web-scale diffusion models to robotics. *IEEE Robotics and Automation Letters*, 8(7):3956–3963.
- Karamcheti, S., Nair, S., Chen, A. S., Kollar, T., Finn, C., Sadigh, D., and Liang, P. (2023). Language-driven representation learning for robotics. *arXiv preprint arXiv:2302.12766*.

- Kareer, S., Patel, D., Punamiya, R., Mathur, P., Cheng, S., Wang, C., Hoffman, J., and Xu, D. (2025). Egomimic: Scaling imitation learning via egocentric video. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13226–13233. IEEE.
- Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., et al. (2024). Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*.
- Kim, M. J., Finn, C., and Liang, P. (2025). Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al. (2024). Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*.
- Lee, J., Duan, J., Fang, H., Deng, Y., Liu, S., Li, B., Fang, B., Zhang, J., Wang, Y. R., Lee, S., et al. (2025). Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*.
- Li, B., Zhang, H., Zhang, K., Guo, D., Zhang, Y., Zhang, R., Li, F., Liu, Z., and Li, C. (2024a). Llava-next: What else influences visual instruction tuning beyond data?
- Li, B., Zhang, Y., Guo, D., Zhang, R., Li, F., Zhang, H., Zhang, K., Zhang, P., Li, Y., Liu, Z., et al. (2024b). Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*.
- Li, Z., Chen, G., Liu, S., Wang, S., VS, V., Ji, Y., Lan, S., Zhang, H., Zhao, Y., Radhakrishnan, S., et al. (2025). Eagle 2: Building post-training data strategies from scratch for frontier vision-language models. *arXiv preprint arXiv:2501.14818*.
- Li, Z., Xie, Y., Shao, R., Chen, G., Jiang, D., and Nie, L. (2024c). Optimus-1: Hybrid multimodal memory empowered agents excel in long-horizon tasks. *Advances in neural information processing systems*, 37:49881–49913.
- Li, Z., Yang, B., Liu, Q., Ma, Z., Zhang, S., Yang, J., Sun, Y., Liu, Y., and Bai, X. (2024d). Monkey: Image resolution and text label are important things for large multi-modal models. In *proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26763–26773.
- Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A. (2023). Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE.
- Lin, K., Agia, C., Migimatsu, T., Pavone, M., and Bohg, J. (2023). Text2motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47(8):1345–1365.
- Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M., and Le, M. (2022). Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*.
- Liu, H., Li, C., Li, Y., and Lee, Y. J. (2024). Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26296–26306.
- Liu, H., Li, C., Wu, Q., and Lee, Y. J. (2023). Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916.
- Liu, X., Gong, C., and Liu, Q. (2022). Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*.
- Lynch, C., Wahid, A., Tompson, J., Ding, T., Betker, J., Baruch, R., Armstrong, T., and Florence, P. (2023). Interactive language: Talking to robots in real time. *IEEE Robotics and Automation Letters*.
- Mandlekar, A., Booher, J., Spero, M., Tung, A., Gupta, A., Zhu, Y., Garg, A., Savarese, S., and Fei-Fei, L. (2019). Scaling robot supervision to hundreds of hours with roboturk: Robotic manipulation dataset through human reasoning and dexterity. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1048–1055. IEEE.
- Mandlekar, A., Zhu, Y., Garg, A., Booher, J., Spero, M., Tung, A., Gao, J., Emmons, J., Gupta, A., Orbay, E., et al. (2018). Roboturk: A crowdsourcing platform for robotic skill learning through imitation. In *Conference on Robot Learning*, pages 879–893. PMLR.
- Nair, S., Rajeswaran, A., Kumar, V., Finn, C., and Gupta, A. (2022). R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*.
- O’Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandlekar, A., Jain, A., et al. (2024). Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE.

- Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., and Levine, S. (2025). Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*.
- Shaw, K., Bahl, S., and Pathak, D. (2023). Videodex: Learning dexterity from internet videos. In *Conference on Robot Learning*, pages 654–665. PMLR.
- Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zoutine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., et al. (2025). Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*.
- Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., and Garg, A. (2022). Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*.
- Singh, S., Yadav, A., Jain, J., Shi, H., Johnson, J., and Desai, K. (2024). Benchmarking object detectors with coco: A new path forward. In *European Conference on Computer Vision*, pages 279–295. Springer.
- Tan, S., Dou, K., Zhao, Y., and Krähenbühl, P. (2025). Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*.
- Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. (2023). Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Team, G. R., Abeyruwan, S., Ainslie, J., Alayrac, J.-B., Arenas, M. G., Armstrong, T., Balakrishna, A., Baruch, R., Bauza, M., Blokzijl, M., et al. (2025). Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*.
- Team, O. M., Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al. (2024). Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*.
- Tian, X., Gu, J., Li, B., Liu, Y., Wang, Y., Zhao, Z., Zhan, K., Jia, P., Lang, X., and Zhao, H. (2024). Drivevlm: The convergence of autonomous driving and large vision-language models. *arXiv preprint arXiv:2402.12289*.
- Tian, Y., Yang, Y., Xie, Y., Cai, Z., Shi, X., Gao, N., Liu, H., Jiang, X., Qiu, Z., Yuan, F., et al. (2025). Interdata-a1: Pioneering high-fidelity synthetic data for pre-training generalist policy. *arXiv preprint arXiv:2511.16651*.
- Vecerik, M., Doersch, C., Yang, Y., Davchev, T., Aytar, Y., Zhou, G., Hadsell, R., Agapito, L., and Scholz, J. (2024). Robotap: Tracking arbitrary points for few-shot visual imitation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5397–5403. IEEE.
- Walke, H. R., Black, K., Zhao, T. Z., Vuong, Q., Zheng, C., Hansen-Estruch, P., He, A. W., Myers, V., Kim, M. J., Du, M., et al. (2023). Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, pages 1723–1736. PMLR.
- Wang, P., Bai, S., Tan, S., Wang, S., Fan, Z., Bai, J., Chen, K., Liu, X., Wang, J., Ge, W., et al. (2024). Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*.
- Wang, Y., Li, X., Wang, W., Zhang, J., Li, Y., Chen, Y., Wang, X., and Zhang, Z. (2025). Unified vision-language-action model. *arXiv preprint arXiv:2506.19850*.
- Wen, C., Lin, X., So, J., Chen, K., Dou, Q., Gao, Y., and Abbeel, P. (2023). Any-point trajectory modeling for policy learning. *arXiv preprint arXiv:2401.00025*.
- Wu, H., Jing, Y., Cheang, C., Chen, G., Xu, J., Li, X., Liu, M., Li, H., and Kong, T. (2023). Unleashing large-scale video generative pre-training for visual robot manipulation. *arXiv preprint arXiv:2312.13139*.
- Wu, P., Shentu, Y., Yi, Z., Lin, X., and Abbeel, P. (2024). Gello: A general, low-cost, and intuitive teleoperation framework for robot manipulators. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12156–12163. IEEE.
- Wu, S., Liu, X., Xie, S., Wang, P., Li, X., Yang, B., Li, Z., Zhu, K., Wu, H., Liu, Y., et al. (2025). Robocoin: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*.
- Wu, W., Lu, F., Wang, Y., Yang, S., Liu, S., Wang, F., Zhu, Q., Sun, H., Wang, Y., Ma, S., et al. (2026). A pragmatic vla foundation model. *arXiv preprint arXiv:2601.18692*.
- Yang, J., Glossop, C., Bhorkar, A., Shah, D., Vuong, Q., Finn, C., Sadigh, D., and Levine, S. (2024). Pushing the limits of cross-embodiment learning for manipulation and navigation. *arXiv preprint arXiv:2402.19432*.
- Yang, Y., Zeng, S., Lin, T., Chang, X., Qi, D., Xiao, J., Liu, H., Chen, R., Chen, Y., Huo, D., et al. (2026). Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*.

- Ye, J., Gao, N., Yang, S., Zheng, J., Wang, Z., Chen, Y., Chen, P., Chen, Y., Liu, S., and Jia, J. (2026). StarVLA- α : Reducing complexity in vision-language-action systems. *arXiv preprint arXiv:2604.11757*.
- Ye, Q., Yu, Z., Shao, R., Xie, X., Torr, P., and Cao, X. (2024a). Cat: Enhancing multimodal large language model to answer questions in dynamic audio-visual scenarios. In *European Conference on Computer Vision*, pages 146–164. Springer.
- Ye, S., Jang, J., Jeon, B., Joo, S., Yang, J., Peng, B., Mandlekar, A., Tan, R., Chao, Y.-W., Lin, B. Y., et al. (2024b). Latent action pretraining from videos. *arXiv preprint arXiv:2410.11758*.
- Yu, T., Xiao, T., Stone, A., Tompson, J., Brohan, A., Wang, S., Singh, J., Tan, C., Peralta, J., Ichter, B., et al. (2023). Scaling robot learning with semantically imagined experience. *arXiv preprint arXiv:2302.11550*.
- Yuan, W., Duan, J., Blukis, V., Pumacay, W., Krishna, R., Murali, A., Mousavian, A., and Fox, D. (2024). Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*.
- Zhang, B., Li, J., Shen, J., Cai, Y., Zhang, Y., Chen, Y., Dai, J., Ji, J., and Yang, Y. (2025). Vla-arena: An open-source framework for benchmarking vision-language-action models. *arXiv preprint arXiv:2512.22539*.
- Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. (2023). Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*.
- Zheng, J., Li, J., Wang, Z., Liu, D., Kang, X., Feng, Y., Zheng, Y., Zou, J., Chen, Y., Zeng, J., et al. (2025). X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*.
- Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., et al. (2023). Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR.

A Additional Related Work

Generalist Robot Policies and Data Pretraining

The emergence of foundation models Bommasani et al. (2021) has catalyzed a shift from task-specific controllers to generalist robotic policies. Early approaches often deploy pre-trained models as high-level semantic planners that interface with low-level, robot-specific skills Zitkovich et al. (2023); Driess et al. (2023); Huang et al. (2022); Liang et al. (2023); Lin et al. (2023); Singh et al. (2022). To overcome the hierarchy’s rigidity, Vision-Language-Action (VLA) models have gained prominence by fine-tuning backbones such as Eagle-2 Li et al. (2025) or PaliGemma Beyer et al. (2024) directly on embodied data for end-to-end optimization Black et al. (2024); Zitkovich et al. (2023); Bjorck et al. (2025); Kim et al. (2024); Yang et al. (2026); Team et al. (2025). Modern VLA architectures frequently incorporate advanced generation techniques like flow-matching Lipman et al. (2022); Liu et al. (2022) and action chunking Zhao et al. (2023). While some frameworks use Mixture-of-Experts to bridge vision and action Black et al. (2024), others utilize cross-attention or embodiment-specific projectors to facilitate multi-robot transfer across latent Team et al. (2024).

The performance of these models is fundamentally driven by data scaling. While high-fidelity trajectories from teleoperation Aldaco et al. (2024); Mandlekar et al. (2018, 2019); Wu et al. (2024) have been scaled into massive datasets with thousands of hours of experience Bu et al. (2025); Black et al. (2024); Brohan et al. (2022); Walke et al. (2023); Lynch et al. (2023); O’Neill et al. (2024), the high cost of collection remains a bottleneck. Consequently, alternative sources such as instrumented human trajectories Chi et al. (2024); Fang et al. (2024) and internet-scale human video datasets Damen et al. (2020); Grauman et al. (2022) are increasingly leveraged. These non-robotic sources serve either as a substrate for representation pretraining Karamcheti et al. (2023); Wu et al. (2023) or as a means of joint learning through motion-based intermediate representations Bharadhwaj et al. (2024b); Ye et al. (2024b). In contrast to prior efforts that often rely on proprietary training stacks, our work systematically explores the nuances of learning from a diverse mixture of public robot data, human videos, and synthetic samples.

Cross-task and cross-embodiment generalization Achieving robust generalization across diverse environments, tasks, and embodiments is a central objective in robot learning. A prevalent strategy involves leveraging human video data to bootstrap policies, either via general representation learning Nair et al. (2022); Bhateja et al. (2023), direct supervision through human motion cloning Shaw et al. (2023); Bharadhwaj et al. (2023); Kareer et al. (2025) or the extraction of 2D point tracks as intermediate representations Bharadhwaj et al. (2024a); Vecerik et al. (2024); Wen et al. (2023). Parallel to these data-driven efforts, another line of work integrates Internet-scale foundation models into training or inference pipelines to inherit broad semantic knowledge Kapelyukh et al. (2023); Yu et al. (2023); Jiang et al. (2023). With the emergence of large-scale, cross-embodiment datasets O’Neill et al. (2024), recent research has increasingly focused on enhancing transfer learning between heterogeneous robot platforms Doshi et al. (2024); Yang et al. (2024).

Vision Language Models Modern Vision-Language Models (VLMs) Chen et al. (2024a); Liu et al. (2024); Dai et al. (2023); Bai et al. (2025); Li et al. (2024d); Wang et al. (2024) have transitioned from task-specific designs toward unified frameworks capable of general-purpose multimodal reasoning Li et al. (2024c); Ye et al. (2024a). Typically comprising a vision encoder, a projector, and a Large Language Model (LLM) backbone Liu et al. (2024); Wang et al. (2024), these models unify diverse tasks as textual outputs. This shift is driven by visual instruction tuning Liu et al. (2023); Chen et al. (2024c), which grants VLMs the flexibility to follow open-ended commands and generalize to out-of-distribution scenarios. Recent advancements further push these boundaries through dynamic resolution Bai et al. (2025), unified image-video processing Li et al. (2024b), and reinforcement-learning-enhanced reasoning Huang et al. (2025), significantly bolstering real-world perception. While VLMs have demonstrated high-level planning in domains like autonomous driving Tian et al. (2024) and GUI interaction Hong et al. (2024), they often remain passive observers. The emerging frontier lies in bridging this "embodiment gap" by extending VLM reasoning into direct physical interaction. By grounding these foundation models into action spaces, Vision-Language-Action (VLA) models leverage the rich semantic and spatial knowledge of VLMs to enable robust robotic manipulation in complex environments.

Table 4: **VLA-Arena results.** Success rate (%) averaged over difficulty levels L0/L1/L2 within each task category. The final average follows the official suite weighting over 11 task suites. Dark blue highlights VLAct and per-column best results; light blue highlights the strongest public baseline by average and per-column second-best results.

Method	Safety	Distractor	Extrap.	Long-H.	Avg.
SmolVLA (Shukor et al., 2025)	16.5	21.0	10.0	24.7	16.3
π_0 -FAST (Pertsch et al., 2025)	34.1	39.0	4.2	20.7	25.6
GR00T-N1.6 (Bjorck et al., 2025)	32.8	40.7	16.7	10.3	27.8
Qwen3-VL- π (Bai et al., 2025)	38.3	45.3	22.4	25.3	34.1
UniVLA (Wang et al., 2025)	45.5	41.3	31.1	22.0	38.7
OpenVLA (Kim et al., 2024)	41.9	43.0	36.7	26.7	39.3
OpenVLA-OFT (Kim et al., 2025)	43.2	52.3	30.4	26.7	39.9
π_0 (Black et al., 2024)	49.7	43.7	32.7	31.3	42.3
$\pi_{0.5}$ (Intelligence et al., 2025)	51.5	54.0	31.1	29.0	44.3
Qwen3-VL-OFT	36.1	40.0	24.7	32.7	33.4
VLAct	60.9	66.0	39.3	33.0	53.4

B Additional Main Results

B.1 Additional Results on Franka

Results on VLA-Arena. Table 4 reports success on VLA-Arena across four behavioral generalization axes. VLAct achieves the best score on every axis and reaches **53.4%** overall. Compared to Qwen3-VL-OFT, our VLAct improves the average score by **20.0%** despite using the same backbone family and downstream OFT head. This large gap again isolates the effect of the representation-centric pre-training recipe rather than the base VLM or the fine-tuning head.

VLAct also outperforms the strongest public baseline, $\pi_{0.5}$, by 9.1 points overall, with especially large gains on *Safety* and *Distractor*, 9.4 and 12.0 points, respectively. Since all methods are fine-tuned on the same downstream task data, the improvement directly demonstrates the effectiveness of VLAct as a pretrained VLA backbone: it provides a substantially stronger starting point for downstream policy learning and yields better generalization across challenging behavioral settings.

B.2 Additional Results on Agilex

Results on DOMINO. Table 5 reports results on DOMINO, a dynamic manipulation benchmark with moving objects and changing scene states. VLAct-OFT achieves the best performance on both metrics, reaching **18.50** SR and **34.20** MS. Compared with Qwen3VL-OFT, the strongest baseline with the same Qwen3-VL backbone family and OFT downstream head, VLAct improves SR by **7.64** points and MS by 3.71 points. Since both models are fine-tuned under the same downstream setting, the improvement highlights the benefit of VLAct as a pretrained backbone for dynamic bimanual manipulation. Together, the RoboTwin 2.0 and DOMINO results show that VLAct is effective across both visually randomized static manipulation and temporally evolving dynamic manipulation.

C Effectiveness of Shallow-Layer Protection During Pre-training

C.1 Motivation and Design

Our VLA-oriented backbone is initialized from a VLM trained on broad vision-language data. This initialization already contains useful low-level visual features, spatial cues, and early vision-language alignment. In contrast, VLA pre-training data is much narrower: robot trajectories often come from fixed camera viewpoints, repeated manipulation scenes, and embodiment-specific action correlations. If all layers are updated end-to-end during VLA pre-training, gradients from this narrow distribution can shift the backbone away from its general VLM representation before the model has learned robust action-conditioned features.

Shallow-layer protection is designed to reduce this representation drift. As illustrated in Fig. 6, lower layers tend to preserve broad visual and spatial information, while deeper layers focus more on semantic and task-

Table 5: **DOMINO dynamic manipulation results.** We evaluate one policy on all 35 DOMINO dynamic manipulation tasks under the clean dynamic setting. DOMINO reports Success Rate (SR) and Manipulation Score (MS). Dark blue highlights VLAAct-OFT; light blue highlights the strongest baseline.

Model	Backbone	SR $\uparrow$	MS $\uparrow$
OpenVLA	Llama-2	1.54	6.10
RDT-1B	DiT-1B	5.34	17.71
π_0	PaliGemma	8.17	23.96
π_0 -FAST	PaliGemma	3.54	20.87
$\pi_{0.5}$	PaliGemma	9.63	26.17
InternVLA-M1	InternVL	5.40	27.57
OpenVLA-OFT	Llama-2	9.06	24.06
Qwen3VL-OFT	Qwen3-VL	<u>10.86</u>	<u>30.49</u>
VLAAct-OFT	Qwen3-VL	18.50	34.20

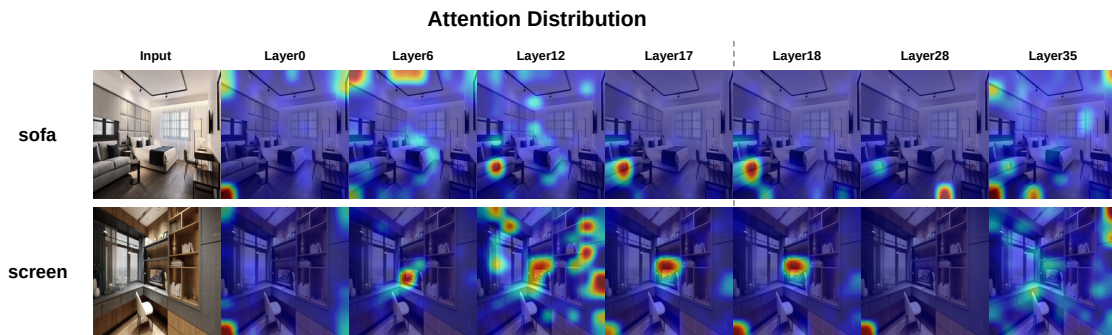


Figure 6: **Layer-wise attention visualization.** Lower layers attend more broadly to visual and spatial information, while deeper layers focus more on semantic and task-relevant regions. This motivates freezing the vision encoder and lower LLM layers during VLA pre-training to reduce representation drift under limited robot data.

relevant regions. We therefore freeze the entire vision encoder and the lower half of the LLM layers during VLA pre-training. This protects the general perceptual and early alignment features inherited from the VLM, while leaving the upper LLM layers and action heads trainable for instruction following, task semantics, and action-conditioned reasoning. During downstream fine-tuning, we unfreeze the full model so that all parameters can adapt to the target task and embodiment.

C.2 Ablation Results

Table 6 reports the ablation. Updating the full backbone during VLA pre-training gives weaker downstream performance, with 78.9% on LIBERO-Plus and 77.1% on RoboTwin2.0. Freezing only the vision encoder improves performance to 81.3 and 79.3%, showing that protecting visual features is already helpful. Freezing both the vision encoder and the lower half of the LLM performs best, reaching 82.6% on LIBERO-Plus and 80.5% on RoboTwin2.0, improving over full-backbone updating by 3.7% and 3.4%, respectively. These results support the role of shallow-layer protection in our recipe: it preserves the general VLM representation during pre-training, while leaving enough upper-layer capacity for action-aware adaptation.

D VLM Co-training for Representation Preservation

D.1 Why Mix Auxiliary Data During VLA Pre-training?

Our first design goal is to preserve the general foundation-model representation inherited from the VLM backbone while adapting it to robot control. During action-only VLA pre-training, the trainable layers are optimized through a narrow supervision channel: repeated robot scenes, embodiment-specific correlations, and

Table 6: **Ablation on shallow-layer protection.** Freezing the vision encoder and lower LLM layers during VLA pre-training reduces representation drift and improves downstream fine-tuning. Dark blue highlights the full design; light blue highlights the strongest partial variant.

Pre-training update strategy	LIBERO-Plus	RoboTwin2.0
Update full backbone	78.9	77.1
Freeze vision encoder only	81.3	79.3
Freeze vision encoder + lower 1/2 LLM	82.6	80.5

sparse action targets. If used alone, this signal can make the backbone over-specialize to the robot trajectory distribution and weaken the semantic, visual, and language coverage that made the original VLM useful.

We therefore mix auxiliary non-action data into VLA pre-training. The goal is not simply to add more samples, but to provide representation-preserving supervision outside the action channel. Robot trajectories teach action-conditioned control, while auxiliary foundation-model data keeps the trainable layers closer to their original operating regime and exposes them to more diverse feature updates. Thus, auxiliary co-training acts as both a representation anchor and a source of feature diversity.

Among the auxiliary sources, image captions play the central role. Detailed captions are close to the dominant supervision used to train the original VLM and provide dense semantic gradients over objects, attributes, relations, and scene context. Grounding and spatial-QA data further constrain localized and spatial representations. We additionally include pure language instruction data as an off-domain diagnostic source: because it is not directly tied to robot perception or control, any gain from this source would suggest that the benefit of auxiliary co-training is not only task-specific transfer, but also representation preservation and diversification.

At every pre-training step, each minibatch contains both robot-trajectory samples and auxiliary samples. We optimize the action loss together with an auxiliary VLM cross-entropy loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{action}} + 0.5 \mathcal{L}_{\text{VLM-CE}}.$$

D.2 Auxiliary Data Mixture

We study five types of auxiliary supervision: image captioning, bounding-box question answering (BBox-QA), point question answering (Point-QA), spatial question answering (Spatial-QA), and pure language instruction data. These sources provide different constraints on the backbone representation. Caption data preserves broad semantic grounding; BBox-QA and Point-QA strengthen localized visual grounding; Spatial-QA targets spatial relations and viewpoint reasoning; pure language instruction data supplies off-domain, non-action supervision on the language side of the backbone. Table 7 summarizes the processed auxiliary datasets used in our study.

Table 7: **Auxiliary data used during VLA pre-training.** The mixture covers captioning, grounding, spatial reasoning, and pure language instruction supervision.

Type	Dataset
Image Caption	LLaVA-ReCap-CC3M (Li et al., 2024a) LLaVA OneVision (Li et al., 2024b)
BBox-QA	RefCOCO (Chen et al., 2025a) COCO-ReM (Singh et al., 2024)
Point-QA	PixMo-Points (Deitke et al., 2025) RoboPoint (Yuan et al., 2024)
Spatial-QA	SenseNova-SI-800K (Cai et al., 2025)
Pure Language Instruction	Nemotron-SFT-Instruction-Following-Chat-v2 ()

Image caption data. We use image caption data as the primary semantic anchor for the VLM representation. LLaVA-ReCap-CC3M (Li et al., 2024a) provides detailed recaptions of web images, preserving broad visual coverage while replacing short or noisy alt-text with richer descriptions. ShareGPT4V (Chen et al., 2024b) provides high-quality descriptive captions that emphasize object identity, attributes, spatial relations, world

knowledge, and visual details. These properties make caption data especially suitable for preserving the backbone’s original vision-language representation during robot pre-training.

Grounding data. BBox-QA and Point-QA data provide localized visual-language supervision. Ref-COCO (Chen et al., 2025a) and COCO-ReM (Singh et al., 2024) are converted into bounding-box question-answering conversations, with coordinates normalized to the 0–1000 convention used by the Qwen3-VL family. PixMo-Points (Deitke et al., 2025) and RoboPoint (Yuan et al., 2024) provide point-based grounding signals. RoboPoint is particularly relevant to robotics because its supervision targets spatial affordance prediction from image-instruction pairs. For efficiency, we filter overly large images and unusually long samples, and restrict each grounding sample to at most 10 boxes or points.

Spatial-QA data. SenseNova-SI-800K (Cai et al., 2025) provides image-conditioned conversations designed to strengthen spatial reasoning. The data covers relative positions, directions, viewpoints, and multi-image spatial context. We keep only single-image cases to control memory cost and remove unusually long conversations for training stability.

We inspect samples from each processed data source to verify the converted format and supervision signal. Representative examples are shown in Fig. 7.

Pure language instruction data. We also include an off-domain auxiliary source from the NVIDIA Nemotron instruction-following dataset, *nvidia/Nemotron-SFT-Instruction-Following-Chat-v2*. This dataset is text-only and targets open-ended chat and instruction-following supervision, rather than visual grounding or robot control. We include it as a control-like auxiliary signal: if pure language supervision improves downstream VLA performance, then the benefit of auxiliary co-training cannot be explained solely by direct transfer from vision-language or robotics-related tasks. Instead, such a gain supports our representation-level interpretation: non-action supervision can help preserve the backbone’s foundation-model operating regime and diversify feature updates during VLA pre-training.

D.3 Effect of Different Auxiliary Co-training Data

We compare different auxiliary supervision types while keeping the robot data, backbone, training recipe, and total pre-training budget fixed. Fig. 8 reports the LIBERO-Plus results. Relative to the robot-only baseline, all auxiliary sources improve downstream performance, indicating that non-action supervision helps produce a stronger VLA-oriented backbone.

Image-caption data gives the strongest single-source gain, improving the baseline from 75.0 to 82.6. This is consistent with its role as the main representation-preservation signal: detailed captions are close to the original VLM pre-training distribution and provide dense semantic supervision over objects, attributes, relations, and scene context. Spatial-QA, Point-QA, and BBox-QA also improve performance, suggesting that spatial reasoning and localized grounding provide useful complementary constraints on the learned representation.

Pure language instruction data also improves over the robot-only baseline. Since this source is text-only and not directly related to robot perception, visual grounding, or control, its gain is especially informative. It suggests that auxiliary co-training does not help only through direct transfer from VLM or robotics-related tasks. Instead, non-action supervision can help preserve the backbone’s foundation-model representation and make feature updates more diverse than action-only training.

The mixed-data setting reaches 82.5, slightly below caption-only co-training. Since the total number of pre-training steps is fixed, mixing multiple auxiliary sources reduces the sampling frequency of image-caption data, which is the strongest single source in this ablation. This likely pulls the mixed result slightly below caption-only co-training, rather than indicating that data mixing itself is harmful.

Overall, this ablation supports the representation-preservation role of auxiliary co-training. Caption data provides the strongest single anchor, while grounding, spatial, and pure language instruction data provide additional gains. These results justify using auxiliary data not as generic extra supervision, but as a targeted mechanism for preserving and enriching the backbone representation during VLA pre-training.

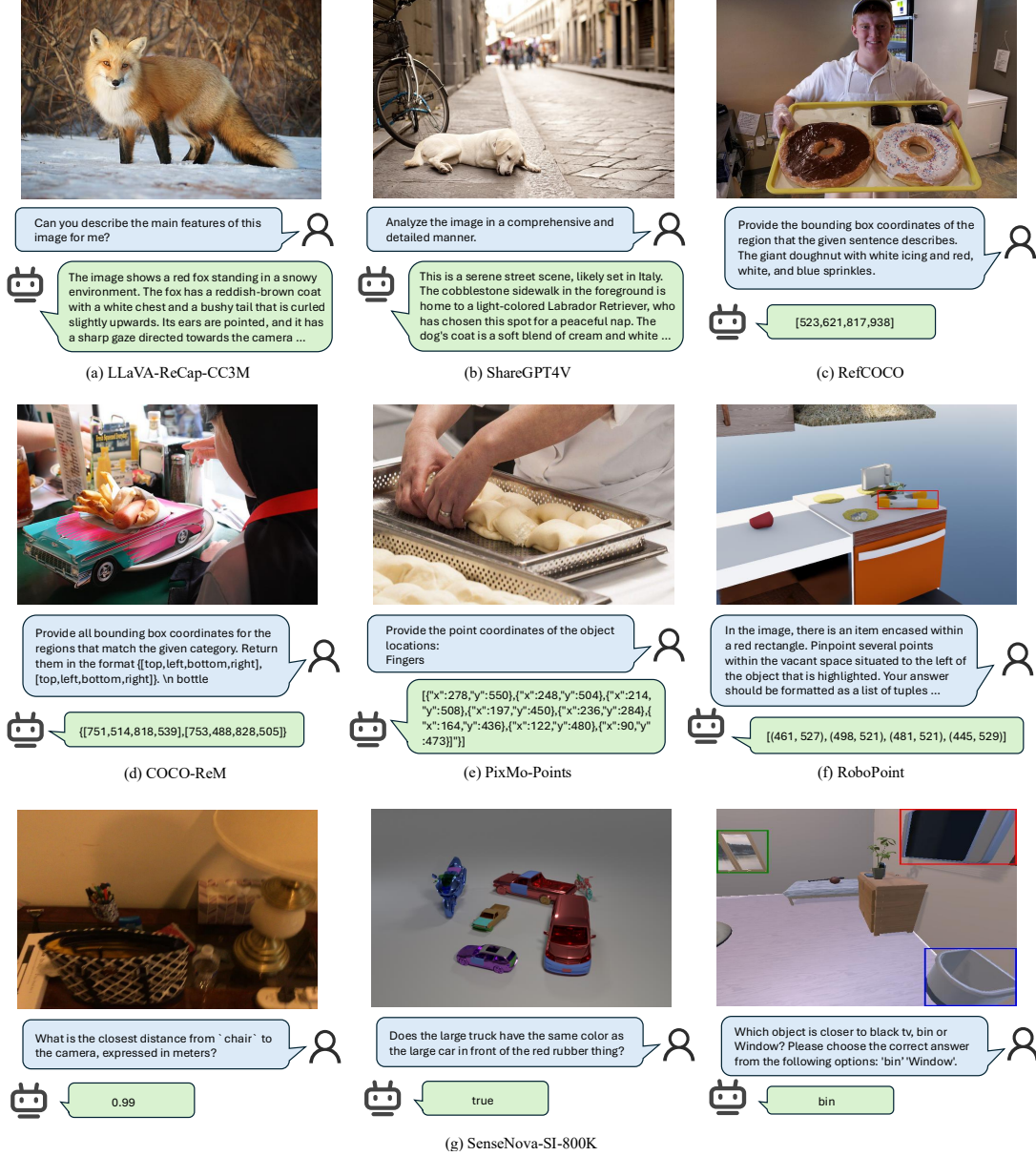


Figure 7: **Examples from the processed VLM data mixture.** We show representative samples from captioning, grounding, and spatial-reasoning datasets used during VLA pre-training.

E Representation Effects of Head-Diverse Pre-training

E.1 Head-Diverse Pre-training Prevents Decoder Lock-in

The pilot study suggests that single-head continuous pre-training can reduce the reusability of the backbone with a different downstream action head. We refer to this tendency as *decoder lock-in*: the backbone may organize action-relevant features in a form that is effective for the pre-training decoder, but less accessible to alternative decoders introduced during fine-tuning. This is a concern for building a reusable VLA-oriented backbone, since downstream users may adopt different action heads depending on the embodiment, control frequency, action horizon, or deployment constraints.

Table 8 examines this issue using PI as the downstream fine-tuning head. Without additional continuous pre-training, PI fine-tuning reaches 60.5. After OFT-only pre-training, PI fine-tuning drops to 55.1, suggesting that the OFT pre-trained representation is not necessarily more compatible with the PI action head. In contrast,

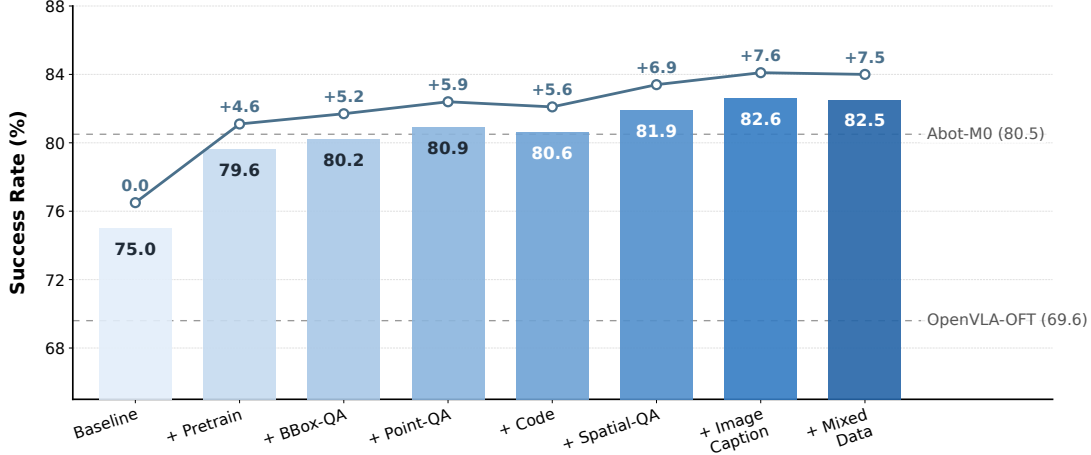


Figure 8: **Effect of auxiliary co-training data.** We compare different non-action supervision sources under the same robot data, model, and pre-training budget. Caption data gives the strongest single-source gain, while grounding, spatial-QA, and pure language instruction data also improve over robot-only training.

Table 8: **Head diversity reduces decoder lock-in.** PI fine-tuning degrades after OFT-only pre-training, but improves after OFT+GR00T pre-training even though PI is excluded during pre-training. Light blue highlights the key unseen-head comparison; dark blue marks the full configuration with direct PI exposure.

Pre-training heads	Seen PI in pre-training	PI FT	Δ vs. scratch
None	–	60.5	–
OFT	No	55.1	–5.4
OFT + GR00T	No	63.1	+2.6
OFT + PI + GR00T	Yes	78.4	+17.9

when the backbone is pre-trained with two continuous heads, OFT and GR00T, PI fine-tuning improves to 63.1, even though PI is not used during pre-training. The improvement is modest, but the comparison between OFT-only and OFT+GR00T is informative: adding a second pre-training head changes the unseen PI fine-tuning result from below the from scratch baseline to slightly above it.

When PI is also included during pre-training, PI fine-tuning reaches 78.4. This result should be interpreted separately, since the downstream head is now directly supervised during pre-training. It confirms that direct head exposure provides a strong compatibility benefit, but the more relevant comparison for decoder lock-in is between OFT-only and OFT+GR00T. Overall, this pilot study suggests that head diversity can reduce over-specialization to a single decoder and improve the accessibility of the learned representation to an unseen downstream head.

E.2 Does Head Diversity Also Improve Same-head Adaptation?

The main motivation for head-diverse pre-training is to improve compatibility across action heads. We further examine whether this comes at the cost of same-head adaptation, where the downstream fine-tuning head matches one of the heads used during pre-training. Ideally, a more head-agnostic representation should remain competitive when the same head is reused.

Table 9 compares three settings: fine-tuning without additional continuous pre-training, fine-tuning from a matched single-head pre-trained backbone, and fine-tuning from the full head-diverse pre-trained backbone. Head-diverse pre-training improves over the corresponding single-head baselines for OFT, PI, and GR00T, with gains of 1.7, 3.0, and 3.5 points, respectively. These gains are relatively moderate compared with the direct benefit of pre-training itself, but they indicate that multi-head supervision does not harm same-head adaptation in our setting.

This result is consistent with the representation-level view in Sec. 3.3. When the backbone is trained with multiple decoders, it must expose action information in a form that can be read by several action

Table 9: **Same-head adaptation under head-diverse pre-training.** Head-diverse pre-training improves downstream performance even when the fine-tuning head matches one of the pre-training heads. Within each row, blue and lighter-blue cells mark the best and second-best results.

Fine-tune head	From scratch	Single-head pre-train	Head-diverse pre-train	Δ vs. single
OFT	61.7	78.8	80.5	+1.7
PI	60.5	75.4	78.4	+3.0
GR00T	51.2	71.7	75.2	+3.5

parameterizations. This reduces over-specialization to any single decoder and can produce action features that are more useful even when the same head is reused.

Summary. These diagnostics support the role of head diversity as a representation-compatibility mechanism. Single-head continuous pre-training can lock the backbone into one decoder geometry, making alternative downstream heads difficult to optimize. Adding multiple heads during pre-training reduces this lock-in by requiring the shared latent representation to support more than one action head. The same representation also improves same-head adaptation in our experiments, but the central objective is broader: to obtain a reusable VLA backbone that can support diverse downstream action heads.

F Unified Joint Space and Wrap-Aware Loss

F.1 Unified joint space by wrapping.

Absolute joint angles are naturally defined on a periodic space rather than on the Euclidean line. In particular, each revolute joint angle should be represented within the canonical range $[-\pi, \pi]$. Angles outside this interval do not correspond to new physical joint states; they are merely alternative parameterizations of equivalent poses. For example, $\pi + \epsilon$ and $-\pi + \epsilon$ describe nearly identical joint configurations, but their raw numerical values differ by almost 2π . If such equivalent angles are kept as different supervision targets, the model is forced to produce different outputs for the same underlying physical state, making it harder to learn a consistent action representation. Therefore, before training, we wrap all absolute joint-angle dimensions into $[-\pi, \pi]$:

$$a_{\text{wrap}} = (a + \pi) \bmod 2\pi - \pi. \quad (1)$$

This data-side wrapping constructs a unified joint space, where equivalent joint states are mapped to a consistent numerical representation.

Wrap-aware loss. Although data-side wrapping ensures that target actions lie in the canonical range, a standard regression loss can still produce an incorrect distance measure near the wrap boundary. For instance, when the target angle is close to $-\pi$ and the prediction is close to $+\pi$, the two angles are physically close, but the naive residual $\hat{a} - a$ is close to 2π . To correct this artifact, we further wrap the residual between the predicted and target actions before computing the regression loss. Given a predicted action $\hat{a}$ and a target action a on a revolute dimension, we define

$$\delta_{\text{wrap}} = ((\hat{a} - a) + \pi) \bmod 2\pi - \pi, \quad (2)$$

and compute the wrap-aware loss with a standard L1 penalty:

$$\mathcal{L}_{\text{wrap}} = |\delta_{\text{wrap}}|. \quad (3)$$

We add this term to the original training objective of each action head. Specifically, for a head h , we denote its original action-learning objective as $\mathcal{L}_{\text{action}}^{(h)}$. This objective is the standard loss used by the corresponding head architecture, such as direct action regression for OFT, diffusion training loss for GR00T, or flow-matching loss for PI. The final objective for head h is therefore

$$\mathcal{L}_{\text{total}}^{(h)} = \mathcal{L}_{\text{action}}^{(h)} + \mathcal{L}_{\text{wrap}}^{(h)}, \quad (4)$$

where $\mathcal{L}_{\text{wrap}}^{(h)}$ is computed on the final action prediction of head h . For OFT, this corresponds to its direct regression output. For GR00T and PI, this corresponds to the final generated action sample after the denoising or flow-based generation process, rather than the intermediate noise prediction or velocity target. The wrap-aware term is applied only to absolute joint-angle dimensions. Non-periodic dimensions, such as gripper commands and delta end-effector translations, are excluded.

Table 10: **Ablation of unified joint space and wrap-aware loss on RoboTwin.** All models are trained under the base setting using 50 tasks with 50 trajectories per task, and evaluated on the Clean split. The pre-training and fine-tuning protocols are kept identical across all variants. Dark blue highlights the full design; light blue highlights the strongest partial variant.

Setting	Unified Joint Space	Wrap Loss	Robotwin
Baseline			75.5
Unified joint space	✓		78.6
Unified joint space + Wrap-aware loss	✓	✓	80.5

F.2 Experiments.

We ablate the effect of data-side joint-angle wrapping and loss-side residual wrapping on RoboTwin under the base setting. Specifically, we train each model using 50 tasks with 50 trajectories per task, and evaluate the trained policy on the Clean evaluation split. For a fair comparison, all variants use the same pre-training and fine-tuning protocol, and only differ in whether unified joint-space wrapping and wrap-aware loss are enabled. As shown in Table 10, the baseline that directly regresses raw joint angles achieves 75.5% success rate. Introducing the unified joint space improves the performance to 78.6%, suggesting that data-side wrapping reduces ambiguity in the action targets. Adding the wrap-aware loss further improves the success rate to 80.5%, showing that residual wrapping provides a more appropriate distance measure near the angular boundary.

G Details of Data Cleaning

Robot Platforms and Action Spaces. We train our models on two robot platforms: a Franka single-arm platform and an AgileX dual-arm platform. For the AgileX dual-arm setting, we train on AgileX data from InternData-A1 and RoboCoin, and evaluate the trained models on RoboTwin. The AgileX action space is 14-dimensional, consisting of 12 absolute joint-angle dimensions for the two arms and 2 gripper dimensions. For the Franka single-arm setting, we train on DROID and MolmoAct, where DROID includes both v1.0.0 and v1.0.1, and evaluate on LIBERO-Plus and VLA-Arena. The Franka action space is 7-dimensional, consisting of 6-dimensional delta end-effector actions and 1 gripper dimension.

Unified Cleaning Pipeline. The training data are aggregated from multiple robot datasets collected with different platforms, annotation conventions, control interfaces, action spaces, and preprocessing pipelines. As a result, the raw data contain several sources of heterogeneity and noise. First, some trajectories have invalid or uninformative task names, which cannot provide meaningful language supervision for instruction-conditioned policy learning. Second, some action sequences contain abnormal values with unrealistically large magnitudes, which are likely caused by data collection failures, logging errors, or preprocessing artifacts. Third, even when the trajectories are valid, different datasets may use different action scales, control frequencies, and gripper conventions. Without proper alignment, the model may learn dataset-specific action patterns rather than a unified action representation that generalizes across datasets.

To address these issues, we apply a unified data cleaning pipeline before training. The pipeline aims to remove unreliable supervision signals while preserving useful trajectories as much as possible. It mainly consists of two stages. First, we remove trajectories or samples with invalid task names, including `unknown_task`, `n/a`, `none`, `no action`, whitespace-only task names, and empty task names. Such annotations do not provide meaningful language supervision and may introduce ambiguity during instruction-conditioned policy learning. Second, we filter action chunks that contain an excessive number of abnormal actions with unusually large magnitudes. This step removes severe action artifacts while avoiding overly aggressive trajectory-level filtering.

Delta End-Effector Action Alignment. For delta end-effector actions, different datasets may be collected at different frame rates, while the physical robot motion speed is generally comparable across datasets. To reduce discrepancies caused by different FPS values, we rescale each delta action by the corresponding FPS and convert it to a unified per-second representation. An action step is marked as invalid if its absolute translational velocity exceeds 0.5 or its absolute rotational velocity exceeds 1.0.

Instead of discarding an entire trajectory whenever abnormal actions appear, we adopt a step-level masking strategy. Specifically, invalid action steps are masked out during training, so that a small number of noisy actions does not remove an otherwise useful trajectory. For each action chunk $\mathcal{C}$, we compute the invalid-step

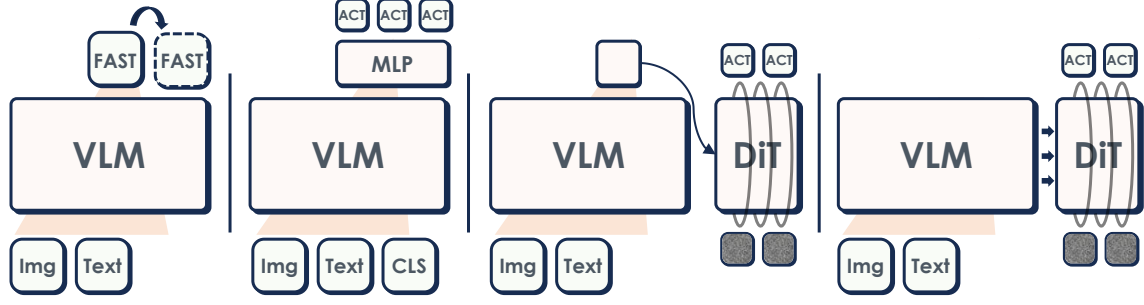


Figure 9: Comparison of representative VLA action heads studied in this work. FAST preserves the VLM’s autoregressive interface by discretizing action chunks into action tokens. OFT attaches a lightweight MLP head to VLM action features and predicts continuous actions in parallel. PI and GR00T use a separate DiT-style motor module conditioned on VLM features to generate continuous action chunks through flow matching, with GR00T using a more explicitly decoupled dual-system design.

ratio:

$$r_c = \frac{1}{|\mathcal{C}|} \sum_{t \in \mathcal{C}} \mathbb{I}[\text{step } t \text{ is invalid}].$$

We discard the whole chunk only when $r_c > 0.5$, since such chunks are dominated by unreliable supervision signals. After filtering, we normalize the translational and rotational components by 0.5 and 1.0, respectively. This maps delta end-effector actions from different datasets into a shared and numerically stable action space.

Absolute Joint-Angle Action Alignment. For absolute joint-angle actions, we first remove values outside the range $[-2\pi, 2\pi]$, as such values are likely caused by data collection errors or preprocessing artifacts. For the remaining valid actions, we wrap each joint angle into the range $[-\pi, \pi]$:

$$\theta_{\text{wrap}} = ((\theta + \pi) \bmod 2\pi) - \pi.$$

This wrapping operation resolves equivalent angular representations and ensures that all joint-angle actions follow a consistent convention. After this step, joint-angle actions from different datasets naturally lie in the same angular space, and therefore we do not apply additional normalization to them.

Gripper Action Normalization. For gripper actions on both robot platforms, we first remove extreme outliers and then apply min-max normalization separately for each dataset. This maps gripper values to continuous values in $[0, 1]$, providing a consistent representation across datasets despite differences in gripper hardware, control interfaces, and recording conventions.

H Details of Action Heads

H.1 Representative Action-Head Families

We briefly summarize the four representative action-head families used in our study. They share the same high-level role: mapping VLM representations to robot action chunks. Their main difference lies in the action interface: FAST uses discrete autoregressive action tokens, OFT uses direct continuous regression, and PI/GR00T use flow-matching-based continuous action generation.

FAST: autoregressive discrete action tokens. FAST converts a continuous action chunk into a sequence of discrete action tokens. Let

$$z_{1:M} = \text{Tok}_{\text{FAST}}(A_t)$$

denote the token sequence obtained from the continuous action chunk. The VLA model is trained with the standard next-token prediction objective

$$\mathcal{L}_{\text{FAST}} = - \sum_{m=1}^M \log p_{\theta}(z_m \mid H_t, z_{<m}).$$

At inference time, the model autoregressively generates action tokens $\hat{z}_{1:M}$ and maps them back to continuous actions through the inverse tokenizer,

$$\hat{A}_t = \text{Tok}_{\text{FAST}}^{-1}(\hat{z}_{1:M}).$$

FAST therefore preserves the native language-modeling form of the VLM: action prediction is reduced to token prediction. Compared with direct per-dimension action binning, FAST-style tokenization compresses the action trajectory before discretization, often by representing action chunks in a frequency-domain or otherwise compact basis. Its advantage is architectural compatibility with autoregressive VLMs; its cost is that continuous control is mediated through a discrete tokenizer and decoded sequentially.

OFT: parallel continuous regression. OFT removes the discrete tokenizer and attaches a lightweight continuous regression head to the VLM. The model appends a fixed set of action query tokens to the backbone sequence. If $h_{t,k}^{\text{act}}$ denotes the hidden state corresponding to the k -th action query, then a small MLP predicts the corresponding action vector:

$$\hat{a}_{t+k} = g_\phi(h_{t,k}^{\text{act}}), \quad k = 0, \dots, K-1.$$

The head is trained with a pointwise regression loss, typically

$$\mathcal{L}_{\text{OFT}} = \frac{1}{Kd_a} \sum_{k=0}^{K-1} \|g_\phi(h_{t,k}^{\text{act}}) - a_{t+k}\|_1.$$

Unlike FAST, OFT predicts the entire action chunk in one forward pass. This makes OFT a simple and efficient probe of whether the backbone representation linearly exposes the information needed for continuous control. The tradeoff is that the output is a point estimate rather than an explicit multimodal action distribution.

PI: flow-matching action expert. PI-style heads model continuous action chunks through conditional flow matching. Instead of directly predicting A_t , the head learns a vector field that transports Gaussian noise into the demonstrated action chunk conditional on the backbone representation. Let $\epsilon \sim \mathcal{N}(0, I)$ and sample a flow time $\tau \in [0, 1]$. With a linear probability path,

$$A_t^\tau = (1 - \tau)\epsilon + \tau A_t,$$

the target velocity is

$$u_t = A_t - \epsilon.$$

The action expert v_ϕ receives the noised action chunk, the flow time, and the backbone representation, and is trained by

$$\mathcal{L}_{\text{PI}} = \mathbb{E}_{A_t, \epsilon, \tau} \left[\|v_\phi(A_t^\tau, \tau; H_t) - (A_t - \epsilon)\|_2^2 \right].$$

At inference time, the head starts from Gaussian noise $A_t^0 \sim \mathcal{N}(0, I)$ and integrates the learned vector field:

$$A_t^{n+1} = A_t^n + \Delta v_\phi(A_t^n, \tau_n; H_t), \quad n = 0, \dots, N-1,$$

returning $\hat{A}_t = A_t^N$. Compared with OFT, PI introduces an iterative generative decoder. This is more expensive than one-shot regression, but it can represent richer continuous action distributions and separates semantic conditioning from motor generation more explicitly.

GR00T: dual-system flow-matching motor module. GR00T also uses flow matching, but places it inside a more explicitly separated dual-system architecture. The vision-language backbone acts as a slow semantic reasoning module, while a DiT-style action module acts as a fast motor-generation module. The backbone encodes images and language into vision-language tokens. These tokens, together with proprioceptive state encodings and noised action encodings, condition a diffusion-transformer action module.

Formally, the training objective has the same flow-matching form:

$$\mathcal{L}_{\text{GR00T}} = \mathbb{E}_{A_t, \epsilon, \tau} \left[\|v_\phi(A_t^\tau, \tau; H_t, s_t, e) - (A_t - \epsilon)\|_2^2 \right],$$

where s_t denotes the robot state and e denotes an embodiment identifier or embodiment-specific adapter. The key distinction from PI is architectural: state and action vectors are embedded into a separate motor module, processed by a DiT-style transformer, and decoded back into the robot’s native action space. Cross-attention to the VLM tokens keeps the motor module conditioned on the scene and instruction while preserving a specialized action-generation pathway.

Table 11: **Effect of unified action representation design.** We compare separate heads, a unified head, and the full unified action representation on RoboTwin and LIBERO-Plus. Dark blue highlights the full design; light blue highlights the unified-head intermediate.

Setting	RoboTwin	LIBERO-Plus
Separate heads	78.5	81.1
Unified head	79.5	81.4
Unified action representation	80.5	82.6

H.2 Unified Action Representation

Although FAST, OFT, PI, and GR00T use different decoding mechanisms, they all ultimately supervise a future action chunk. In our multi-embodiment pre-training, we therefore map each dataset’s native action format into a shared canonical output space before applying the corresponding head-specific objective.

Design For the continuous heads used in our main pre-training recipe, the action head emits a fixed 20-dimensional vector at each action step,

$$\hat{a}_{t+k} \in \mathbb{R}^{20}, \quad k = 0, \dots, K - 1.$$

The dimensions are assigned by embodiment role:

- Dimensions 1–6: the left 6-DoF arm of bimanual embodiments such as AgileX, represented as absolute joint angles.
- Dimensions 7–12: the right 6-DoF arm of bimanual embodiments such as AgileX, represented as absolute joint angles.
- Dimensions 13–18: the 6-DoF arm of single-arm embodiments such as Franka, represented as delta end-effector pose.
- Dimension 19: the shared gripper coordinate, used by Franka’s single gripper and AgileX’s left gripper.
- Dimension 20: the right gripper coordinate of bimanual embodiments.

This layout preserves each corpus’s native low-level action convention: Franka actions use delta end-effector pose in DROID and MolmoAct, while AgileX actions use absolute joint angles in InternData-A1 and RoboCoin. Thus, the unified head design does not force all embodiments into the same controller parameterization. It only aligns output coordinates at the level of embodiment role, allowing the same FAST/OFT/PI/GR00T head families to be instantiated on a common cross-embodiment action representation.

Experiments We evaluate the effect of the unified action representation design under three settings: separate embodiment-specific heads, a unified head without action space alignment, and a unified head with unified action representation. Experiments are conducted on RoboTwin and LIBERO-Plus. As shown in Table 11, simply sharing the action head already improves performance over separate heads, while the fully unified action representation achieves the best results on both benchmarks. This suggests that aligning outputs by embodiment role provide a more reusable and transferable action representation across robots.

I Details of Real-world Experiments

I.1 Hardware and observation.

As shown in Fig. 10, our physical evaluation platform uses stationary, table-mounted Franka Research 3 7-DoF arms (one for single-arm setups, two for dual-arm tasks). The visual system is a multi-view configuration: a single Intel RealSense D435 is fixed as an external camera providing a third-person perspective, and each robot arm carries a wrist-mounted Intel RealSense D405 for precise first-person observations. In line with the input specification of our representation-centric pre-training recipe, all images from external and wrist cameras are resized to 224×224 before being processed by the model.

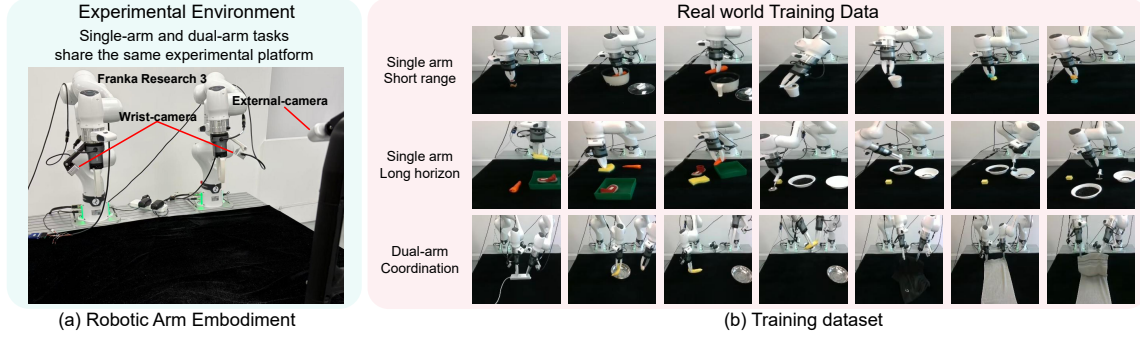


Figure 10: Overviews of our robot setting and real-world training dataset. (a) Robot setting for the experiments. (b) We collect real-world robot trajectories for eleven task suites.

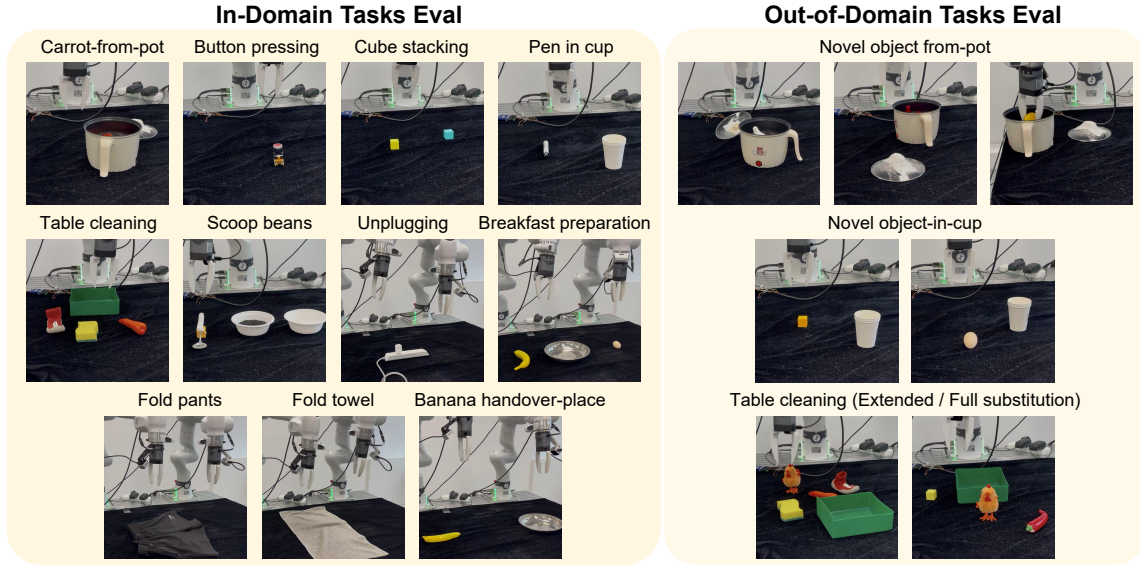


Figure 11: Schematic illustrations of real-robot in-domain and out-of-domain evaluation tasks.

I.2 Detailed Real-World Results and Failure Analysis

Success Rate Calculation Criterion. For real-machine experiments, each task is rolled out for 10 trajectories to calculate the task success rate. The scoring rules vary for different task types. Specifically, for single-arm short-range tasks and dual-arm collaborative tasks, a score of 1 is awarded for a successful task completion and 0 for a failure. For single-arm long-range tasks, the scoring mechanism is based on the number of completed task steps. Taking the table cleaning task as an example, the entire task consists of three independent steps, and each successfully completed step contributes 0.33 points to the total task score.

Single-arm short-horizon tasks. VLAAct achieves an average success rate of 92.5% on in-domain single-arm short-horizon tasks, compared with 77.5% for Qwen3VL-4B-OFT without pre-training. Both models perform well on simpler tasks such as *carrot-from-pot*, where both reach 100% SR, and *button pressing*, where VLAAct reaches 100% SR and the baseline reaches 90% SR. However, the gap becomes larger when the task requires more precise spatial localization. On *cube stacking*, VLAAct reaches 90% SR while the baseline drops to 60%. We observe that the baseline often freezes during inference when the cube starts from positions not well covered by the training distribution. On *pen-in-cup*, VLAAct reaches 80% SR compared with 60% for the baseline. VLAAct can localize the cup even when it is placed near the rear of the table, while the baseline often accumulates errors after failed grasping attempts and enters unstable recovery behaviors.

Short-horizon OOD generalization. VLAAct also generalizes better to unseen objects. For *novel object from-pot*, where the carrot is replaced by egg, pepper, or garlic, VLAAct obtains 90.0% average SR, while

the baseline reaches 73.3%. For *novel object in-cup*, where the target object is replaced by cube or egg, VLAct again reaches 90.0% average SR, compared with 65.0% for the baseline. These results indicate that the pre-trained backbone provides stronger visual-semantic grounding and more stable localization under object-level distribution shift.

Single-arm long-horizon tasks. For long-horizon tasks, VLAct shows a clearer advantage in preserving task logic over multiple manipulation stages. On *table cleaning*, VLAct obtains a weighted success rate 86.6%, compared with 73.3% for the baseline. On *scooping beans*, VLAct reaches 80.0%, whereas the baseline only reaches 33.3%. In this task, VLAct usually completes the intended sequence: grasping the spoon, scooping a bean, and pouring it into the target bowl. By contrast, the baseline frequently skips the scooping phase and directly performs a pouring motion with an empty spoon. In cluttered table-cleaning scenes, the baseline also tends to overlook small objects, while VLAct completes the sequence more fluently.

Long-horizon OOD generalization. We further evaluate long-horizon robustness under two harder OOD settings. In the extended-sequence setting, a novel toy chicken is added to the original table cleaning task. VLAct achieves a weighted success rate of 82.5%, nearly doubling the baseline score of 47.5%. In the full-substitution setting, all objects are replaced by unseen categories, including cube, toy chicken, and pepper. VLAct maintains a score of 83.3%, while the baseline drops to 46.6%. The baseline often misses objects or fails to recover from grasping errors, whereas VLAct maintains the task sequence under both object replacement and sequence extension.

Dual-arm coordination. Across dual-arm coordination tasks, VLAct achieves 72.0% average SR, outperforming the baseline’s 44.0% SR. On *breakfast preparation*, VLAct reaches 90.0% SR compared with 70.0% for the baseline. On *unplugging*, VLAct reaches 80.0% SR compared with 60.0% for the baseline, showing better coordination in the hold-and-pull interaction. The largest gaps appear in deformable-object manipulation. On *fold pants*, VLAct achieves 70.0% SR, outperforming the baseline by 30.0%. The baseline often selects inaccurate grasping points on the fabric, leading to gripper-table collisions or failed lifting. On *fold towel*, VLAct reaches 50.0% SR, while the baseline reaches only 20.0%. The baseline frequently suffers from velocity mismatch between the two arms, causing the towel to slip from one gripper. VLAct produces more synchronized dual-arm motion and more stable contact with deformable objects.

I.3 Detailed Prompts for Each Real-World Robotic Arm Task

Single-arm (Short-horizon)

- **Carrot-from-pot:** Pick the carrot out of the pot and place it on the table.
- **Button pressing:** Press the red button.
- **Cube stacking:** Pick up one cube and stack it on top of another cube.
- **Pen in cup:** Pick up the pen and place it into the paper cup.

Single-arm (Long-horizon)

- **Table cleaning:** Pick up the small objects on the table one by one and place them into the green box.
- **Scoop beans:** First pick up the spoon, then use it to scoop a bean from one bowl, and finally pour the bean into another bowl.

Dual-arm Coordination

- **Unplugging:** One robotic arm holds the socket in place, while the other pulls the plug out and places it on the table.
- **Breakfast preparation:** One robotic arm picks up the banana, while the other picks up the egg; both are then placed on the plate.
- **Banana handover-place:** One robotic arm picks up the banana and hands it to the other arm; the second arm receives it and places it on the plate.
- **Fold pants:** Two robotic arms fold a pair of pants together.
- **Fold towel:** Two robotic arms fold a towel together.

Out-of-Domain (OOD)

- **Novel object from-pot:** Pick the garlic / pepper / egg out of the pot and place it on the table.
- **Novel object in-cup:** Pick up the cube / egg and place it into the paper cup.
- **Table cleaning (extended / full substitution):** Pick up the small objects on the table one by one and place them into the green box.

I.4 Visualization

Detailed Visualization of Training Data for Each Real-World Robotic Arm Task. Fig. 12 shows the visualization of training data for each real-world robotic arm task.

Detailed Visualization of Inference Demos for Each Real-World Robotic Arm Task. Fig. 13 shows the inference demos for Single arm short range real-world task, including In-Domain and Out-of-Domain tasks.

Detailed Visualization of Inference Demos for Each Real-World Robotic Arm Task. Fig. 14 shows the inference demos for Single arm long horizon and Dual arm coordination real-world task, including In-Domain and Out-of-Domain tasks..

J Future Work and Broader Impact.

Limitations. Due to resource constraints, our study focuses on a 4B-scale VLM backbone and does not explore substantially larger models. Larger VLMs may contain stronger visual, spatial, and language priors, and the optimal pre-training recipe may change with model scale.

Broader Impact. We hope this work encourages the community to pay more attention to how VLM backbones should be trained for VLA tasks, rather than treating the backbone as a fixed component inherited from general vision-language pre-training. Future work can study how to build VLA-oriented VLMs that could better generalize across tasks, environments, and embodiments. More broadly, improving the training of reusable VLA backbones may make robot learning less dependent on massive proprietary robot datasets, helping more researchers build generalizable VLA models with accessible data and compute.

Timestep →

Carrot-
from-pot



Button
pressing



Cube
stacking



Pen in
cup

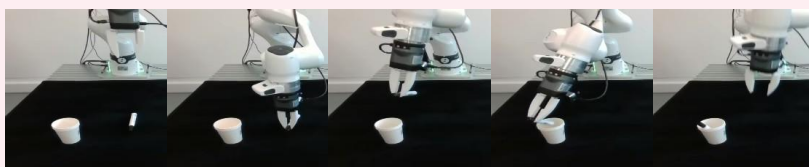
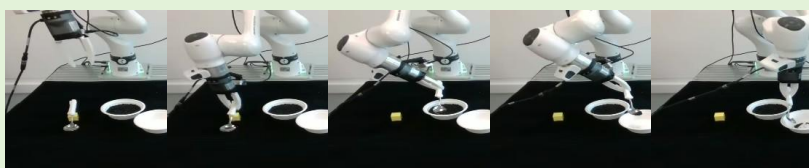


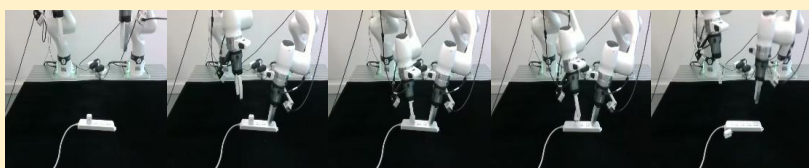
Table
cleaning



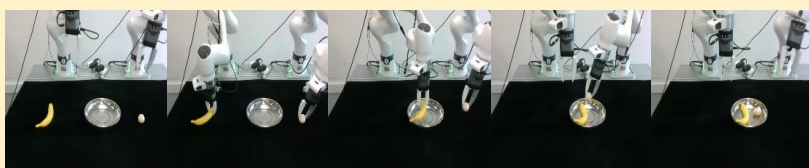
Scoop
beans



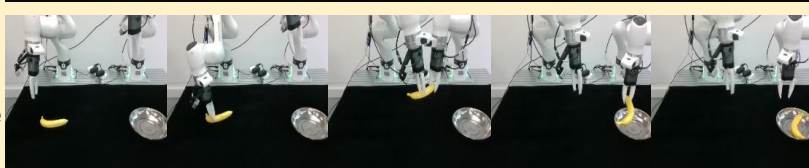
Unplugging



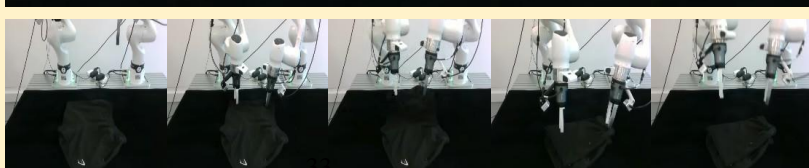
Breakfast
preparation



Banana
handover-place



Fold pants



Fold towel



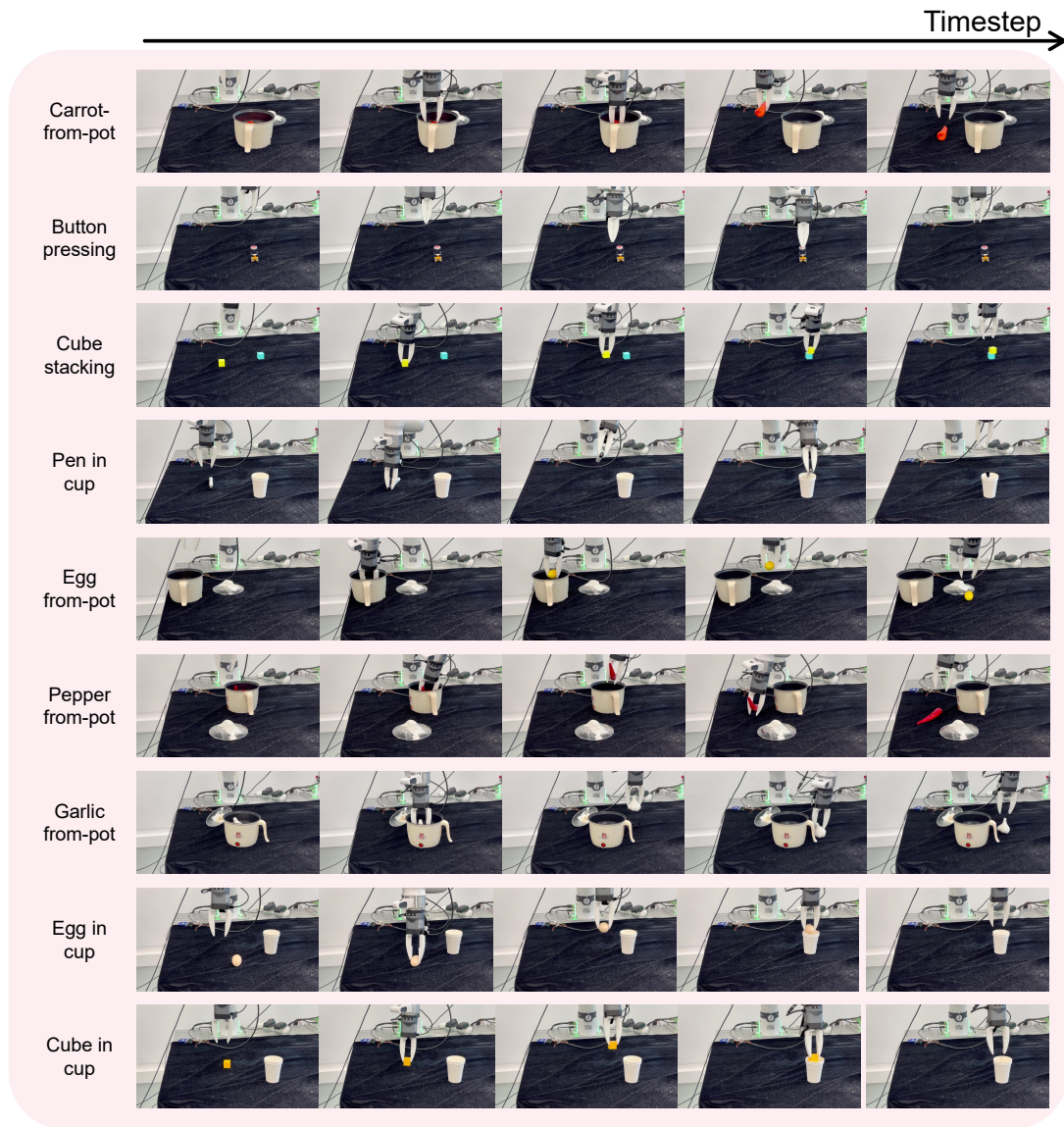


Figure 13: **Detailed visualization for Single arm short range real-world robotic arm task inference demo**
 We show the inference visualization for Single arm short range real-world task, including In-Domain and Out-of-Domain tasks.

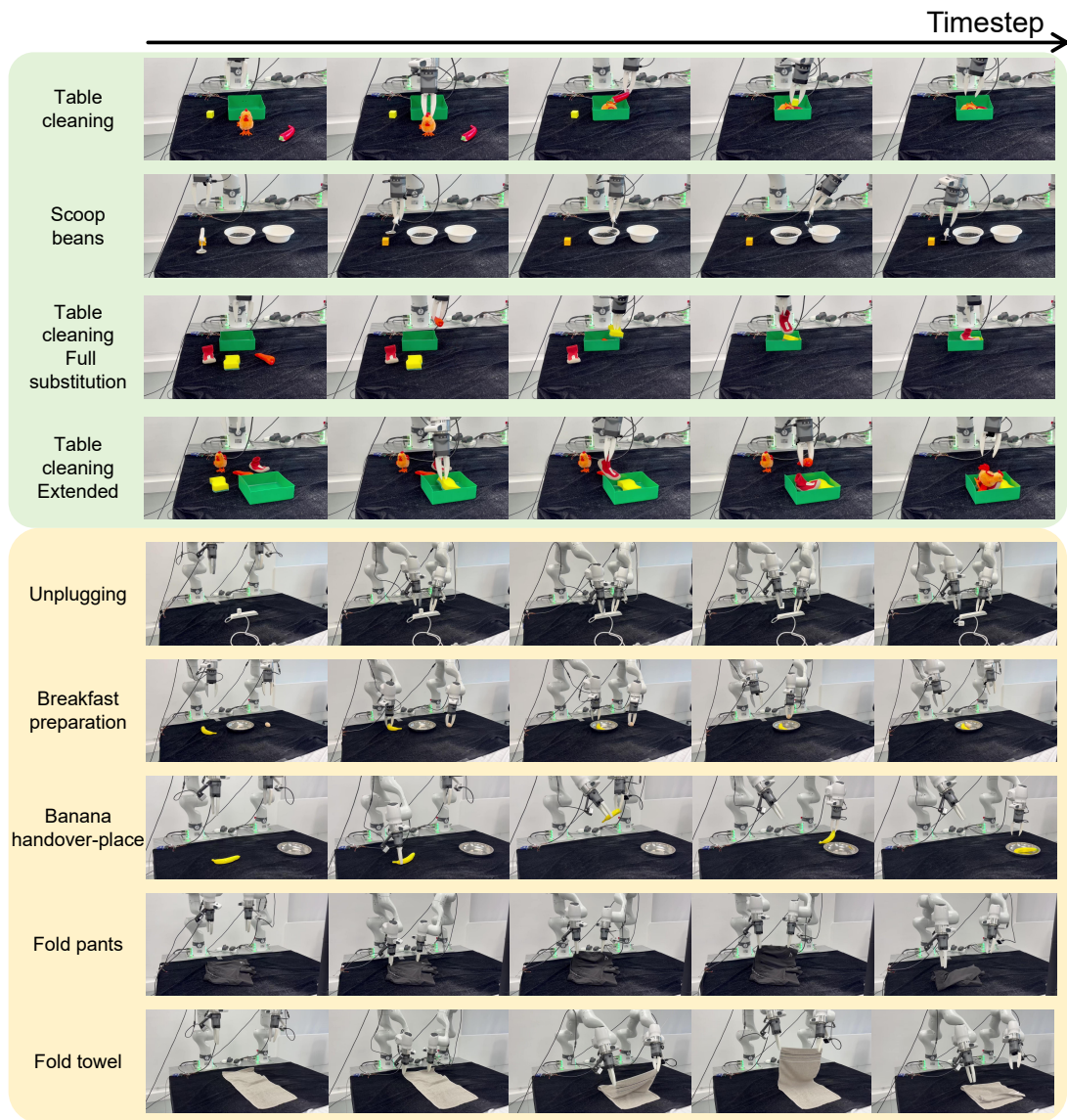


Figure 14: **Detailed visualization for Single arm long horizon and Dual arm coordination real-world robotic arm task inference demo** We show the inference visualization for Single arm long horizon and Dual arm coordination real-world task, including In-Domain and Out-of-Domain tasks.

Authors and Advisors

Authors

- Senqiao Yang[†]
- Chengyao Wang[†]
- Yuxin Chen
- Zixuan Wang
- Longxiang Tang
- Haokun Gui
- Jinhui Ye
- Changsheng Lu
- Xiaoyang Wu
- Mingkang Zhu

Advisors

- Pengguang Chen
- Shu Liu[✉]
- Zhuotao Tian
- Hengshuang Zhao
- Bei Yu
- Jiaya Jia

[†] Project leaders [✉] Correspondence

For questions or suggestions, please feel free to contact us at [✉] yangsenqiao.ai@gmail.com.